

Digital Marketing Campaign Conversion Indicator

- **Project Type** : EDA & Machine Learning
- **Contribution** : Individual
- **Created by** : Rishabh Salian

Project Overview

This project aims to enhance campaign effectiveness in the digital marketing sector by developing a robust machine learning model to accurately predict customer conversions. The model will utilize a comprehensive customer-level dataset containing demographics, campaign attributes, engagement metrics, and prior behavior to forecast the binary Conversion outcome. This data allows for the analysis of which channels and campaign types drive conversions, explores the relationship between engagement and conversion, and enables performance segmentation. You can preview or plot column details to inform your analysis, with the ultimate objective of improving targeting, increasing conversion rates, and maximizing return on advertising spend (ROAS).

Project Summary

This project involved a comprehensive analysis of a digital marketing campaign dataset with 8,000 unique customer records , 1 Key feature and 19 attributes spanning demographics, engagement metrics, ad spend, and campaign characteristics to build a machine learning model for predicting conversions. The goal is to accurately identify likely converters to improve targeting, lift conversion rates, and maximize ROAS across channels like Social, Email, and PPC.

Data Assessment & Cleaning The initial data assessment revealed a mix of data types (object, float64, int64), along with significant data quality issues:

- **Duplicate Records:** 0 duplicate entries were identified.
- **Missing Values:** No missing values are found in the dataset.
- **Outlier Detection:** No outliers are found in the dataset.
- **Datatype Validation:** 2 columns (ClickThroughRate, ConversionRate) was identified with incorrect datatype (object) and corrected to datatype(float).

Data Processing Workflow The project followed a structured data manipulation pipeline:

- **Data Collection:** Initial exploration using functions like head(), info(), describe(), and columns().
- **Data Cleaning & Transformation:** Addressed quality issues by checking outliers, missing & duplicate values. Additionally:
 - The ClickThroughRate & ConversionRate column was converted from an object to a float data type.
 - New categorical columns were engineered for analysis.

Exploratory Data Analysis (EDA) & Visualization A thorough exploratory analysis was conducted following the UBM framework:

- **Univariate Analysis:** Understanding the distribution of individual variables.
- **Bivariate Analysis:** Examining relationships between two variables, covering all combinations (Numerical-Categorical, Numerical-Numerical, Categorical-Categorical).
- **Multivariate Analysis:** Investigating more complex interactions between multiple variables.

Various charts and a correlation matrix were used to uncover key patterns and insights. The goal of this visualization was to translate raw data into actionable business intelligence that could drive growth and improve decision-making.

"Through meticulous data cleaning and multi-faceted exploration, this analysis uncovered the critical factors that influence customers conversion, providing valuable insights to support strategic business development."

Define the Business Objective?

- Explore and Analyze the data to discover important factors that governs the customers rate of conversion and derive insights for improving it's rate.
- Build a Prediction machine learning model which accurately predict customer conversions.

Life cycle of the Project / Table of Content

1. Understanding the Problem Statement

2. Data Collection

- 2.1 Import Data and Required Packages

- 2.2 Dataset Information

3. Data Checks to perform

- 3.1 Check Data Type
- 3.2 Check Duplicates
- 3.3 Check Missing Values
- 3.4 Check the number of unique values of each column
- 3.5 Check statistics of data set
- 3.6 Check various categories present in the different categorical column
- 3.7 Check for Outliers

4. Data Transformation

- 4.1 Creating addition categorical features for better understanding
- 4.2 Updating Categorical features List
- 4.3 Creating Dummies for the New Dataframe

5. Exploratory data analysis

- 5.1 Demographic Features
- 5.2 Marget Segment Features
- 5.3 Customer Engagement Features
- 5.4 Historical Data
- 5.5 Correlation with Every Features
- 5.6 Question And Answers

6. Data Pre-Processing

- 6.1 Cleaning Dataset and make it ready for training model
- 6.2 Check and Correct Outliers
- 6.3 Comparing Every features with the target Value
- 6.4 Encoding Categorical Variables

7. Model Training

- 7.1 Training Base Model
- 7.2 Training Additional Models

- 7.3 Fine Tuning Parameters and Cross Validation

8. Choose best model

- 8.1 Top 3 Model Selection
- 8.2 Model Interpretation

1. Problem Statement

- What if you could pinpoint which prospects are most likely to convert before you spend a single dollar on ads?
- Could smarter targeting across email, social, and PPC lift conversions while cutting wasted spend?
- And what if you could adjust ad budgets in near real time based on predicted conversion likelihood to maximize ROAS?

Using customer demographics, engagement signals, and campaign attributes, this project builds a robust machine learning model to predict conversions, enabling precise audience targeting, optimized channel mix, and smarter budget allocation to improve conversion rates and maximize return on ad spend.

2. Data Collection

- **Dataset Source** : Boston Institute Analytics (BIA)
- **Dimension** : The data consists of 19 column and 8000 rows.

2.1 Import Data and Required Packages

```
In [1]: # Base Libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import squarify
import warnings
warnings.filterwarnings("ignore")

# Import Connectors
import mysql.connector
from sqlalchemy import create_engine
```

Creating Connection engine with MYSQL Database


```
In [2]: # Creating pipeline through sql server
engine = create_engine('mysql+mysqlconnector://root:l4r5a1d7c3@localhost/digital_marketing_campaign')
dm_query = 'select * from dm_campaign'
```

Import & Reading Dataset from SQL Connection as Pandas Dataframe

```
In [3]: # Importing data from the mysql database
df = pd.read_sql(dm_query,engine)

# Backing up the Original data
df_backup = df.copy

# Structuring dataframe and show Top 5 Records
pd.set_option('display.max_columns',None)
pd.set_option('display.width',None)
pd.set_option('display.max_colwidth',None)
df.head(5)
```

```
Out[3]:
```

	CustomerID	Age	Gender	Income	CampaignChannel	CampaignType	AdSpend	ClickThroughRate	ConversionRate	Website
0	8000	56	Female	136912	Social Media	Awareness	6497.87	0.04	0.09	
1	8001	69	Male	41760	Email	Retention	3898.67	0.16	0.18	
2	8002	46	Female	88456	PPC	Awareness	1546.43	0.28	0.08	
3	8003	32	Female	44085	PPC	Conversion	539.53	0.14	0.09	
4	8004	60	Female	83964	PPC	Conversion	1678.04	0.25	0.11	

Shape of the Dataset

```
In [4]: # Showcase the Total Number of Rows and Columns in the Data
print("-"*60)
print(f"Total Rows in Dataset : {df.shape[0]}")
print(f"Total Columns in Dataset : {df.shape[1]}")
print("-"*60)
```

Total Rows in Dataset : 8000
Total Columns in Dataset : 20

2.2 Dataset Information

Personal Features

- **CustomerID:** Unique identifier for each customer.
- **Age:** Age of the customer.
- **Gender:** Gender of the customer (Male/Female).
- **Income:** Annual income of the customer.

Marketing-specific Variables

- **CampaignChannel:** The channel through which the marketing campaign is delivered (Email, Social Media, SEO, PPC, Referral).
- **CampaignType:** Type of the marketing campaign (Awareness, Consideration, Conversion, Retention).
- **AdSpend:** Amount spent on the marketing campaign.
- **ClickThroughRate:** Rate at which customers click on the marketing content.
- **ConversionRate:** Rate at which clicks convert to desired actions (e.g., purchases).
- **AdvertisingPlatform:** Confidential.
- **AdvertisingTool:** Confidential.

Customer Engagement Variables

- **WebsiteVisits:** Number of visits to the website.
- **PagesPerVisit:** Average number of pages visited per session.
- **TimeOnSite:** Average time spent on the website per visit (in minutes).
- **SocialShares:** Number of times the marketing content was shared on social media.
- **EmailOpens:** Number of times marketing emails were opened.
- **EmailClicks:** Number of times links in marketing emails were clicked.

Historical Data

- **PreviousPurchases:** Number of previous purchases made by the customer.
- **LoyaltyPoints:** Number of loyalty points accumulated by the customer.

Target Variable

- **Conversion:** Binary variable indicating whether the customer converted (1) or not (0).

3. Data Checks to perform

- Check Data Type
- Check Duplicates
- Check Missing Values
- Check the number of unique values of each column
- Check statistics of data set
- Check various categories present in the different categorical column
- Check for Outliers

3.1 Check data type

```
In [5]: # Check Null and Dtypes  
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
```

```
RangeIndex: 8000 entries, 0 to 7999
```

```
Data columns (total 20 columns):
```

#	Column	Non-Null Count	Dtype
0	CustomerID	8000 non-null	int64
1	Age	8000 non-null	int64
2	Gender	8000 non-null	object
3	Income	8000 non-null	int64
4	CampaignChannel	8000 non-null	object
5	CampaignType	8000 non-null	object
6	AdSpend	8000 non-null	float64
7	ClickThroughRate	8000 non-null	object
8	ConversionRate	8000 non-null	object
9	WebsiteVisits	8000 non-null	int64
10	PagesPerVisit	8000 non-null	float64
11	TimeOnSite	8000 non-null	timedelta64[ns]
12	SocialShares	8000 non-null	int64
13	EmailOpens	8000 non-null	int64
14	EmailClicks	8000 non-null	int64
15	PreviousPurchases	8000 non-null	int64
16	LoyaltyPoints	8000 non-null	int64
17	AdvertisingPlatform	8000 non-null	object
18	AdvertisingTool	8000 non-null	object
19	Conversion	8000 non-null	int64

```
dtypes: float64(2), int64(10), object(7), timedelta64[ns](1)
```

```
memory usage: 1.2+ MB
```

```
In [6]: # Fixing the Incorrect Data Type
df['ClickThroughRate'] = df['ClickThroughRate'].astype(float)
df['ConversionRate'] = df['ConversionRate'].astype(float)
df.info()
```

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 8000 entries, 0 to 7999
Data columns (total 20 columns):
 #   Column                Non-Null Count  Dtype
---  -
 0   CustomerID            8000 non-null   int64
 1   Age                   8000 non-null   int64
 2   Gender                8000 non-null   object
 3   Income                8000 non-null   int64
 4   CampaignChannel        8000 non-null   object
 5   CampaignType           8000 non-null   object
 6   AdSpend               8000 non-null   float64
 7   ClickThroughRate       8000 non-null   float64
 8   ConversionRate         8000 non-null   float64
 9   WebsiteVisits          8000 non-null   int64
10   PagesPerVisit          8000 non-null   float64
11   TimeOnSite             8000 non-null   timedelta64[ns]
12   SocialShares           8000 non-null   int64
13   EmailOpens             8000 non-null   int64
14   EmailClicks            8000 non-null   int64
15   PreviousPurchases      8000 non-null   int64
16   LoyaltyPoints          8000 non-null   int64
17   AdvertisingPlatform     8000 non-null   object
18   AdvertisingTool         8000 non-null   object
19   Conversion             8000 non-null   int64
dtypes: float64(4), int64(10), object(5), timedelta64[ns](1)
memory usage: 1.2+ MB

```

- There are 2 feature columns ("ClickThroughRate" & "ConversionRate") with incorrect datatypes
- Changed it to float dtype from object dtype

3.2 Check Duplicates

```

In [7]: # Returning count of duplicated values from the entire dataframe as a aggregate
df.duplicated().sum()

```

```

Out[7]: np.int64(0)

```

There are no duplicate values in the data set

3.3 Check Missing values

```
In [8]: # Checking missing values for each individual columns.  
df.isna().sum()
```

```
Out[8]: CustomerID ..... 0  
Age ..... 0  
Gender ..... 0  
Income ..... 0  
CampaignChannel ..... 0  
CampaignType ..... 0  
AdSpend ..... 0  
ClickThroughRate ..... 0  
ConversionRate ..... 0  
WebsiteVisits ..... 0  
PagesPerVisit ..... 0  
TimeOnSite ..... 0  
SocialShares ..... 0  
EmailOpens ..... 0  
EmailClicks ..... 0  
PreviousPurchases ..... 0  
LoyaltyPoints ..... 0  
AdvertisingPlatform ..... 0  
AdvertisingTool ..... 0  
Conversion ..... 0  
dtype: int64
```

There are no missing values in the data set

3.4 Check the number of unique values of each column

```
In [9]: # Finding out count of every Unique values of each and every coulms  
df.nunique()
```

```
Out[9]: CustomerID ..... 8000
        Age ..... 52
        Gender ..... 2
        Income ..... 7789
        CampaignChannel ..... 5
        CampaignType ..... 4
        AdSpend ..... 7967
        ClickThroughRate ..... 30
        ConversionRate ..... 20
        WebsiteVisits ..... 50
        PagesPerVisit ..... 900
        TimeOnSite ..... 806
        SocialShares ..... 100
        EmailOpens ..... 20
        EmailClicks ..... 10
        PreviousPurchases ..... 10
        LoyaltyPoints ..... 3983
        AdvertisingPlatform ..... 1
        AdvertisingTool ..... 1
        Conversion ..... 2
        dtype: int64
```

3.5 Check statistics of data set

```
In [10]: # Checking Stats Scores for every numeric features in the dataset
         df.describe()
```

Out[10]:

	CustomerID	Age	Income	AdSpend	ClickThroughRate	ConversionRate	WebsiteVisits	PagesPerVisit
count	8000.00000	8000.000000	8000.000000	8000.000000	8000.000000	8000.000000	8000.000000	8000.000000
mean	11999.50000	43.625500	84664.196750	5000.944856	0.154927	0.104454	24.751625	5.549275
std	2309.54541	14.902785	37580.387945	2838.038185	0.084200	0.054994	14.312269	2.607385
min	8000.00000	18.000000	20014.000000	100.050000	0.010000	0.010000	0.000000	1.000000
25%	9999.75000	31.000000	51744.500000	2523.217500	0.080000	0.060000	13.000000	3.300000
50%	11999.50000	43.000000	84926.500000	5013.440000	0.150000	0.100000	25.000000	5.530000
75%	13999.25000	56.000000	116815.750000	7407.987500	0.230000	0.150000	37.000000	7.840000
max	15999.00000	69.000000	149986.000000	9997.910000	0.300000	0.200000	49.000000	10.000000

3.6 Check various categories present in the different categorical column

In [11]:

```
# Identify categorical columns based on no. of unique values in columns
df[df.columns[:-1]].nunique()[df.nunique() < 10]
```

Out[11]: Gender 2
CampaignChannel 5
CampaignType 4
AdvertisingPlatform 1
AdvertisingTool 1
dtype: int64

In [12]:

```
print("Categories in 'Gender' variable:")
print(df['Gender'].unique())
print('-'*60)

print("Categories in 'CampaignChannel' variable:")
print(df['CampaignChannel'].unique())
print('-'*60)

print("Categories in 'CampaignType' variable:")
print(df['CampaignType'].unique())
print('-'*60)
```



```

print("Categories in 'AdvertisingPlatform' variable:")
print(df['AdvertisingPlatform'].unique())
print('-'*60)

print("Categories in 'AdvertisingTool' variable:")
print(df['AdvertisingTool'].unique())
print('-'*60)

```

```

Categories in 'Gender' variable:
['Female' 'Male']
-----

```

```

Categories in 'CampaignChannel' variable:
['Social Media' 'Email' 'PPC' 'Referral' 'SEO']
-----

```

```

Categories in 'CampaignType' variable:
['Awareness' 'Retention' 'Conversion' 'Consideration']
-----

```

```

Categories in 'AdvertisingPlatform' variable:
['IsConfid']
-----

```

```

Categories in 'AdvertisingTool' variable:
['ToolConfid']
-----

```

```

In [13]: # define numerical, categorical & target columns
numeric_features = [feature for feature in df.columns[:-1] if df[feature].dtype != 'O']
categorical_features = [feature for feature in df.columns[:-1] if df[feature].dtype == 'O']
target_input = [df.columns[-1]]

# print columns
print('We have {} numerical features : \n{}'.format(len(numeric_features), numeric_features))
print('-'*60)
print('We have {} categorical features : \n{}'.format(len(categorical_features), categorical_features))
print('-'*60)
print('We have {} target column : \n{}'.format(len(target_input), target_input))
print('-'*60)

```

```
We have 14 numerical features :  
['CustomerID', 'Age', 'Income', 'AdSpend', 'ClickThroughRate', 'ConversionRate', 'WebsiteVisits', 'PagesPerVisit', 'Time  
OnSite', 'SocialShares', 'EmailOpens', 'EmailClicks', 'PreviousPurchases', 'LoyaltyPoints']  
-----
```

We have 5 categorical features :

```
['Gender', 'CampaignChannel', 'CampaignType', 'AdvertisingPlatform', 'AdvertisingTool']  
-----
```

We have 1 target column :

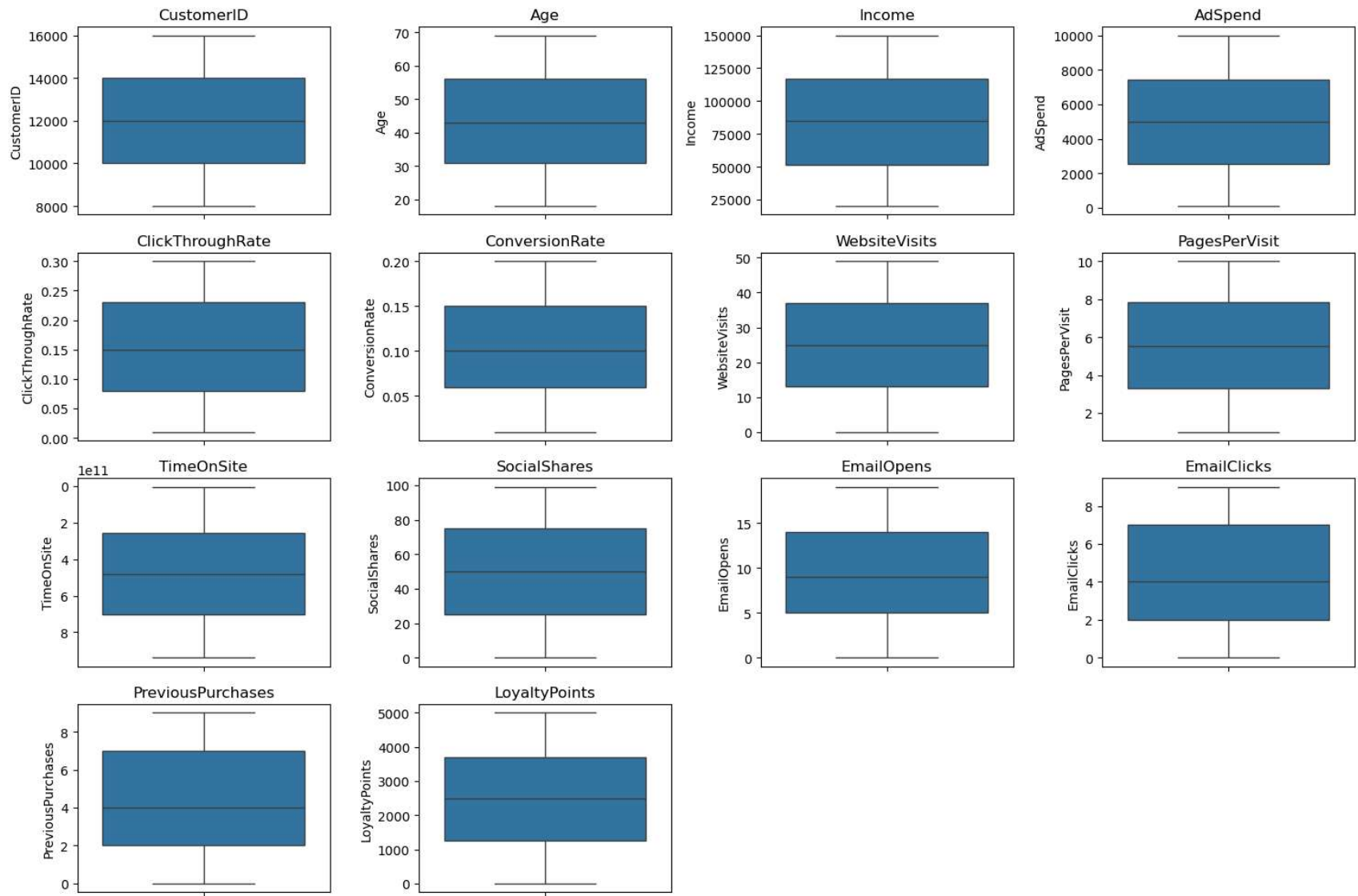
```
['Conversion']  
-----
```

Insights

- From the above analysis we can determine that there are 13 numeric features along with 5 categorical features which can help determine the target values from the target column "Conversion".
- We also checked there is only 1 personal feature and 4 target specific variables among the categorical columns.

3.7 Check for Outliers

```
In [14]: #Outlier Detection with Coxplots  
plt.figure(figsize=(15,10))  
for i, col in enumerate(numeric_features):  
    plt.subplot(4, 4, i+1)  
    sns.boxplot(y=df[col])  
    plt.title(col)  
plt.tight_layout()  
plt.show()
```



There are no outliers located in the dataset

4. Data Transformation

4.1 Creating addition categorical features for better understanding

```
In [15]: # Defining new Queries
dc_query = 'select * from dm_demographic'
```

```

ec_query = 'select * from dm_engagement'

# Importing additional data from the mysql database
dc_df = pd.read_sql(dc_query,engine)
ec_df = pd.read_sql(ec_query,engine)

# Create the new columns
df['Age_Group'] = dc_df['Age_Group']
df['Income_Group'] = dc_df['Income_Group']
df['Spend_level'] = ec_df['AdSpend_Group']
df['Customer_Type'] = ec_df['Pre_Purchase_Group']
df['Loyalty_Rank'] = ec_df['Customer_Loyalty']

# Define the new order of columns
new_column_order = ['Age', 'Age_Group', 'Gender', 'Income', 'Income_Group', 'CampaignChannel', 'CampaignType', 'AdSpend',
                    'ClickThroughRate', 'ConversionRate', 'WebsiteVisits', 'PagesPerVisit',
                    'TimeOnSite', 'SocialShares', 'EmailOpens', 'EmailClicks',
                    'PreviousPurchases', 'Customer_Type', 'Loyalty_Rank', 'LoyaltyPoints', 'AdvertisingPlatform',
                    'AdvertisingTool', 'Conversion']

# Reorder the DataFrame columns
df = df[new_column_order]
df.head(1)

```

Out[15]:

	Age	Age_Group	Gender	Income	Income_Group	CampaignChannel	CampaignType	AdSpend	Spend_level	ClickThroughRa
0	56	Adult	Female	136912	High Income	Social Media	Awareness	6497.87	High Spend	0.0



In [16]:

```

# Showcase the Total Number of Rows and Columns in the New DataFrame
print("-"*60)
print(f"Total Rows in Dataset : {df.shape[0]}")
print(f"Total Columns in Dataset : {df.shape[1]}")
print("-"*60)

```

```

-----
Total Rows in Dataset : 8000
Total Columns in Dataset : 24
-----

```

Insights

- The 5 New Categorical Features ("Age_Group", "Income_Group", "Spend_Level", "Loyalty_Rank" & "Customer_Type") will help better understand the customer demography and features which influence the conversion.
- With this the Dimension of the Data has risen from 19 attributes to 24 attributes for the same 8000 records. This will help the model to train much better.

4.2 Updating Categorical features List

```
In [17]: # Adding the features in the Categorical feature list
categorical_features = [feature for feature in df.columns[:-1] if df[feature].dtype == 'O']
df[categorical_features].head(1)
```

```
Out[17]:
```

	Age_Group	Gender	Income_Group	CampaignChannel	CampaignType	Spend_level	Customer_Type	Loyalty_Rank	Advertisin
0	Adult	Female	High Income	Social Media	Awareness	High Spend	Frequent Buyer	Bronze Tier	

4.3 Creating Dummies for the New Dataframe

```
In [18]: # Let's Convert all categorical features into dummy variables
df_dummies = pd.get_dummies(df)

# Define the new order of columns
new_dummies_column_order = ['Age', 'Age_Group_Youth', 'Age_Group_Adult', 'Age_Group_Senior', 'Gender_Male', 'Gender_Fem',
                             'Income_Group_Low Income', 'Income_Group_Lower Middle Income', 'Income_Group_Middle Income',
                             'Income_Group_Upper Middle Income', 'Income_Group_High Income', 'CampaignChannel_Email', 'Ca',
                             'CampaignChannel_Referral', 'CampaignChannel_SEO', 'CampaignChannel_Social Media', 'Campaig',
                             'CampaignType_Consideration', 'CampaignType_Conversion', 'CampaignType_Retention', 'AdSpend',
                             'Spend_level_Medium Spend', 'Spend_level_High Spend', 'ClickThroughRate', 'Customer_Type_Ne',
                             'Customer_Type_Occasional Buyer', 'Customer_Type_Frequent Buyer', 'ConversionRate', 'Websit',
                             'TimeOnSite', 'SocialShares', 'EmailOpens', 'EmailClicks', 'PreviousPurchases', 'LoyaltyPoi',
                             'Loyalty_Rank_Bronze Tier', 'Loyalty_Rank_Silver Tier', 'Loyalty_Rank_Gold Tier', 'Loyalty_',
                             'AdvertisingPlatform_IsConfid', 'AdvertisingTool_ToolConfid', 'Conversion']

# Reorder the DataFrame columns
df_dummies = df_dummies[new_dummies_column_order]
df_dummies.head(1)
```

Out[18]:

	Age	Age_Group_Youth	Age_Group_Adult	Age_Group_Senior	Gender_Male	Gender_Female	Income	Income_Group_Low Income	Incc
0	56	False	True	False	False	True	136912	False	

5. Exploratory Data Analysis (Visualization)

5.1 Demographic Features

5.1.1 Univariate & Bivariate

Age

In [19]:

```
# Count the Customers of each age
age_counts = df['Age'].value_counts().sort_index()

# Calculate Conversion based on Gender
converted_df = df[df['Conversion'] == 1]
converted_age_counts = converted_df['Age_Group'].value_counts()
not_converted_df = df[df['Conversion'] == 0]
not_converted_age_counts = not_converted_df['Age_Group'].value_counts()

# Conversions for each Age Group
conversion_by_age_group = df.groupby("Age_Group")["Conversion"].sum().sort_values(ascending=False)

# Create a figure with two subplots
fig, axes = plt.subplots(2, 1, figsize=(12, 12))

# Plot 1: Distribution of Customer Age
sns.barplot(x=age_counts.index, y=age_counts.values, color=sns.color_palette('Blues_r')[0], ax=axes[0])
axes[0].set_title('Distribution of Customer Age (Overall)')
axes[0].set_xlabel('Customer Age')
axes[0].set_ylabel('Customer Count')
axes[0].tick_params(axis='x', rotation=90)

# Plot 2: Total Conversions by Age Group
all_age = df['Age_Group'].unique()
converted_age_counts = converted_age_counts.reindex(all_age, fill_value=0)
not_converted_age_counts = not_converted_age_counts.reindex(all_age, fill_value=0)
```

```

age = converted_age_counts.index
converted_values = converted_age_counts.values
not_converted_values = not_converted_age_counts.values

bars_converted = axes[1].barh(age, converted_values, color='#2675c3', label='Converted')
bars_not_converted = axes[1].barh(age, not_converted_values, left=converted_values, color='#ededed', label='Not Convert

axes[1].set_title("Conversion and Non-Conversion by Gender Distribution")
axes[1].set_xlabel("Number of Customers")
axes[1].set_ylabel("Customer Age_Group")
axes[1].legend()

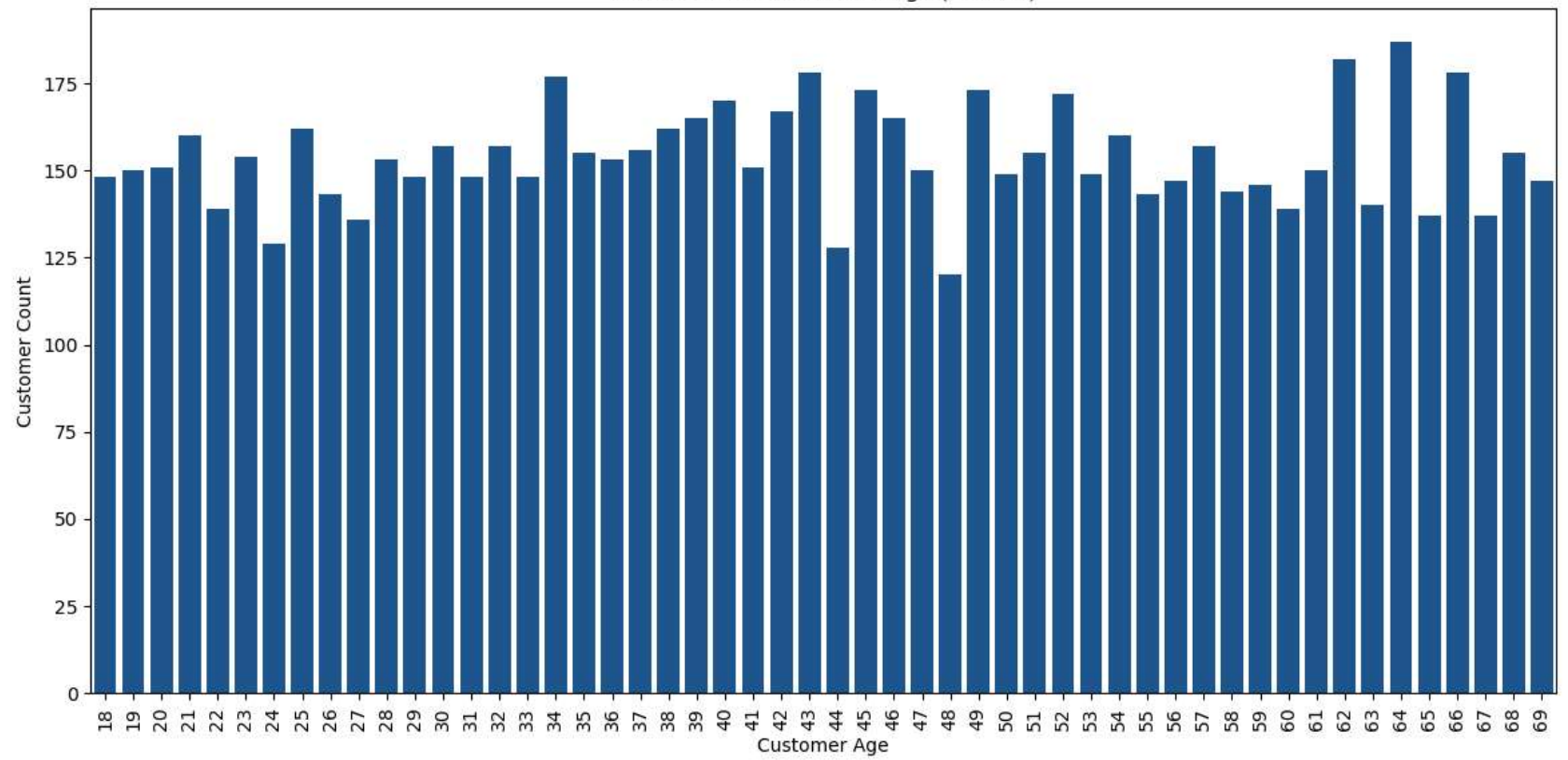
# Add text annotations inside the bars
for bar in bars_converted:
    width = bar.get_width()
    label_text = f'{int(width)}'
    xpos = width - (width * 0.1) # Position slightly inside the bar
    ypos = bar.get_y() + bar.get_height() / 2
    axes[1].text(xpos, ypos, label_text, va='center', ha='right', color='white', fontsize=10)

for bar in bars_not_converted:
    width = bar.get_width()
    label_text = f'{int(width)}'
    xpos = bar.get_x() + width - (width * 0.1) # Position slightly inside the bar
    ypos = bar.get_y() + bar.get_height() / 2
    axes[1].text(xpos, ypos, label_text, va='center', ha='right', color='black', fontsize=10)

plt.tight_layout()
plt.show()

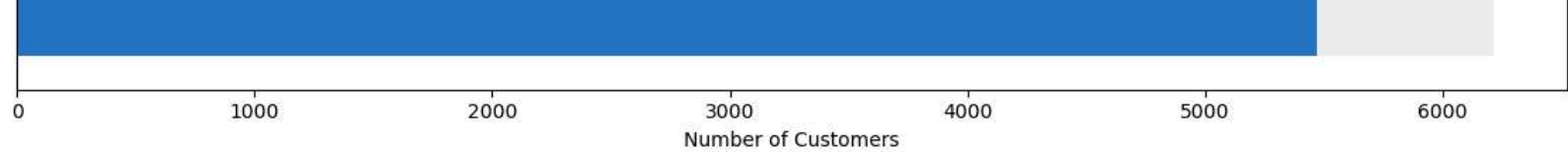
```

Distribution of Customer Age (Overall)



Conversion and Non-Conversion by Gender Distribution





Insights:

- **Overall Age Distribution:** The customer age distribution appears to be relatively uniform across the different age groups (Youth, Adult, Senior), indicating a diverse customer base.
- **Conversion by Age Group:** The 'Adult' age group shows the highest number of conversions, followed by 'Youth' and then 'Senior'. This suggests that marketing efforts might be most effective with the adult demographic, or that this group has a higher natural propensity to convert. Further analysis would be needed to determine if this is due to campaign targeting or inherent characteristics of these age groups.

Gender

```
In [20]: # Calculate overall Gender distribution
overall_gender_counts = df['Gender'].value_counts()

# Calculate Conversion based on Gender
converted_df = df[df['Conversion'] == 1]
converted_gender_counts = converted_df['Gender'].value_counts()
not_converted_df = df[df['Conversion'] == 0]
not_converted_gender_counts = not_converted_df['Gender'].value_counts()

# Create a figure with two subplots
fig, axes = plt.subplots(1, 2, figsize=(12, 6))

# Plot 1: Distribution of Gender
explode = (0.1, 0)
axes[0].pie(overall_gender_counts, labels=overall_gender_counts.index, autopct='%1.1f%%', startangle=90, colors=sns.col
axes[0].set_title('Overall Customer Gender Distribution')
axes[0].axis('equal')

# Plot 2: Total Conversions by Gender
all_genders = df['Gender'].unique()
converted_gender_counts = converted_gender_counts.reindex(all_genders, fill_value=0)
not_converted_gender_counts = not_converted_gender_counts.reindex(all_genders, fill_value=0)
```

```

genders = converted_gender_counts.index
converted_values = converted_gender_counts.values
not_converted_values = not_converted_gender_counts.values

bars_converted = axes[1].barh(genders, converted_values, color='#2675c3', label='Converted')
bars_not_converted = axes[1].barh(genders, not_converted_values, left=converted_values, color='#ededed', label='Not Con

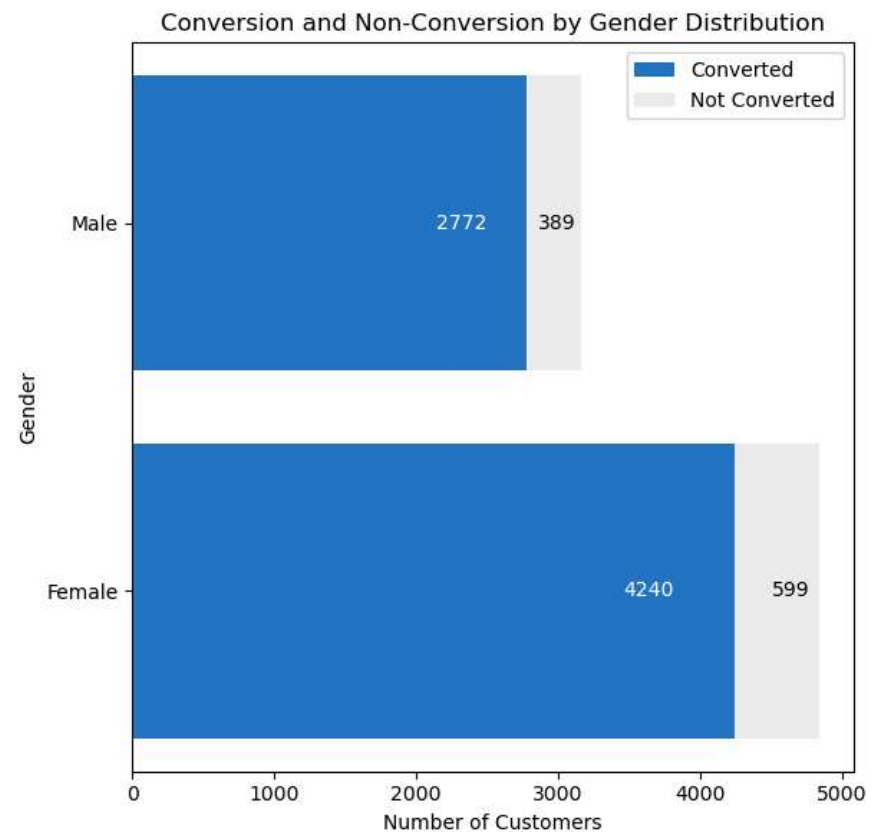
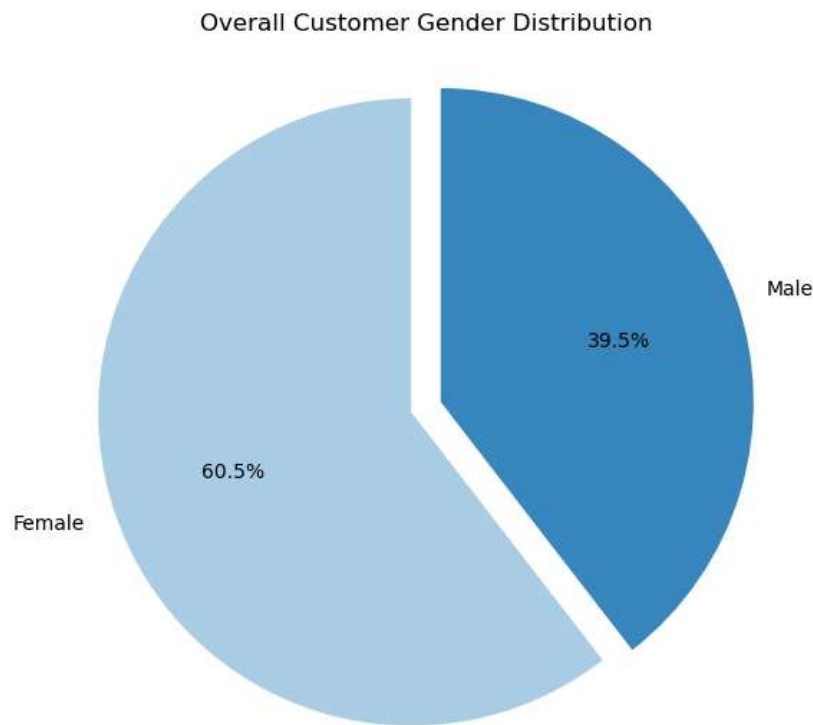
axes[1].set_title("Conversion and Non-Conversion by Gender Distribution")
axes[1].set_xlabel("Number of Customers")
axes[1].set_ylabel("Gender")
axes[1].legend()

# Add text annotations inside the bars
for bar in bars_converted:
    width = bar.get_width()
    label_text = f'{int(width)}'
    xpos = width - (width * 0.1) # Position slightly inside the bar
    ypos = bar.get_y() + bar.get_height() / 2
    axes[1].text(xpos, ypos, label_text, va='center', ha='right', color='white', fontsize=10)

for bar in bars_not_converted:
    width = bar.get_width()
    label_text = f'{int(width)}'
    xpos = bar.get_x() + width - (width * 0.1) # Position slightly inside the bar
    ypos = bar.get_y() + bar.get_height() / 2
    axes[1].text(xpos, ypos, label_text, va='center', ha='right', color='black', fontsize=10)

plt.tight_layout()
plt.show()

```



Insights:

- **Overall Gender Distribution:** The dataset has more female customers than male customers, with approximately 60.5% female and 39.5% male representation.
- **Conversion by Gender:** While the absolute number of female conversions is higher (4240) compared to male conversions (2772), the conversion rate for both genders is almost identical, at around 87.6% for females and 87.7% for males. This indicates that gender itself does not significantly influence the likelihood of conversion, but rather the higher number of female conversions is due to their larger representation in the customer base.

Income

```
In [21]: # Conversions for each Income_Group
conversion_by_income_group = df.groupby("Income_Group")["Conversion"].sum().sort_values(ascending=False)

# Calculate Conversion based on Channel
```

```

converted_df = df[df['Conversion'] == 1]
converted_income_counts = converted_df['Income_Group'].value_counts()
not_converted_df = df[df['Conversion'] == 0]
not_converted_income_counts = not_converted_df['Income_Group'].value_counts()

# Create a figure with two subplots
fig, axes = plt.subplots(2, 1, figsize=(12, 12))

# Plot 1: Distribution of Top Customer Income
ax1 = sns.kdeplot(data=df, x='Income', fill=True, alpha=0.5, linewidth=2, color='steelblue', ax=axes[0])
sns.rugplot(data=df, x='Income', alpha=0.3, height=0.05, ax=axes[0])

# Define stats and color
stats = {'Mean': df['Income'].mean(), 'Median': df['Income'].median(),
        '25th Percentile': df['Income'].quantile(0.25), '75th Percentile': df['Income'].quantile(0.75)}
colors = ['red', 'green', 'orange', 'purple']

for (label, value), color in zip(stats.items(), colors):
    axes[0].axvline(value, color=color, linestyle='--', linewidth=1.5, label=f'{label}: ${value:,.0f}')

axes[0].set_title('Comprehensive Analysis: Customer Income Distribution',
                  fontsize=16, fontweight='bold', pad=20)
axes[0].set_xlabel('Income', fontsize=13)
axes[0].set_ylabel('Density', fontsize=13)
axes[0].grid(True, alpha=0.3)
axes[0].legend(bbox_to_anchor=(1.05, 1), loc='upper left')

# Plot 2: Total Conversions by Income Group
income = df['Income'].unique()
converted_channel_counts = converted_income_counts.reindex(income, fill_value=0)
not_converted_channel_counts = not_converted_income_counts.reindex(income, fill_value=0)

income_group = converted_income_counts.index
converted_values = converted_income_counts.values
not_converted_values = not_converted_income_counts.values

bars_converted = axes[1].barh(income_group, converted_values, color='#2675c3', label='Converted')
bars_not_converted = axes[1].barh(income_group, not_converted_values, left=converted_values, color='#ededed', label='No

axes[1].set_title("Conversion and Non-Conversion by Income Group Distribution")
axes[1].set_xlabel("Number of Customers")
axes[1].set_ylabel("Income Group")
axes[1].legend()

# Add text annotations inside the bars

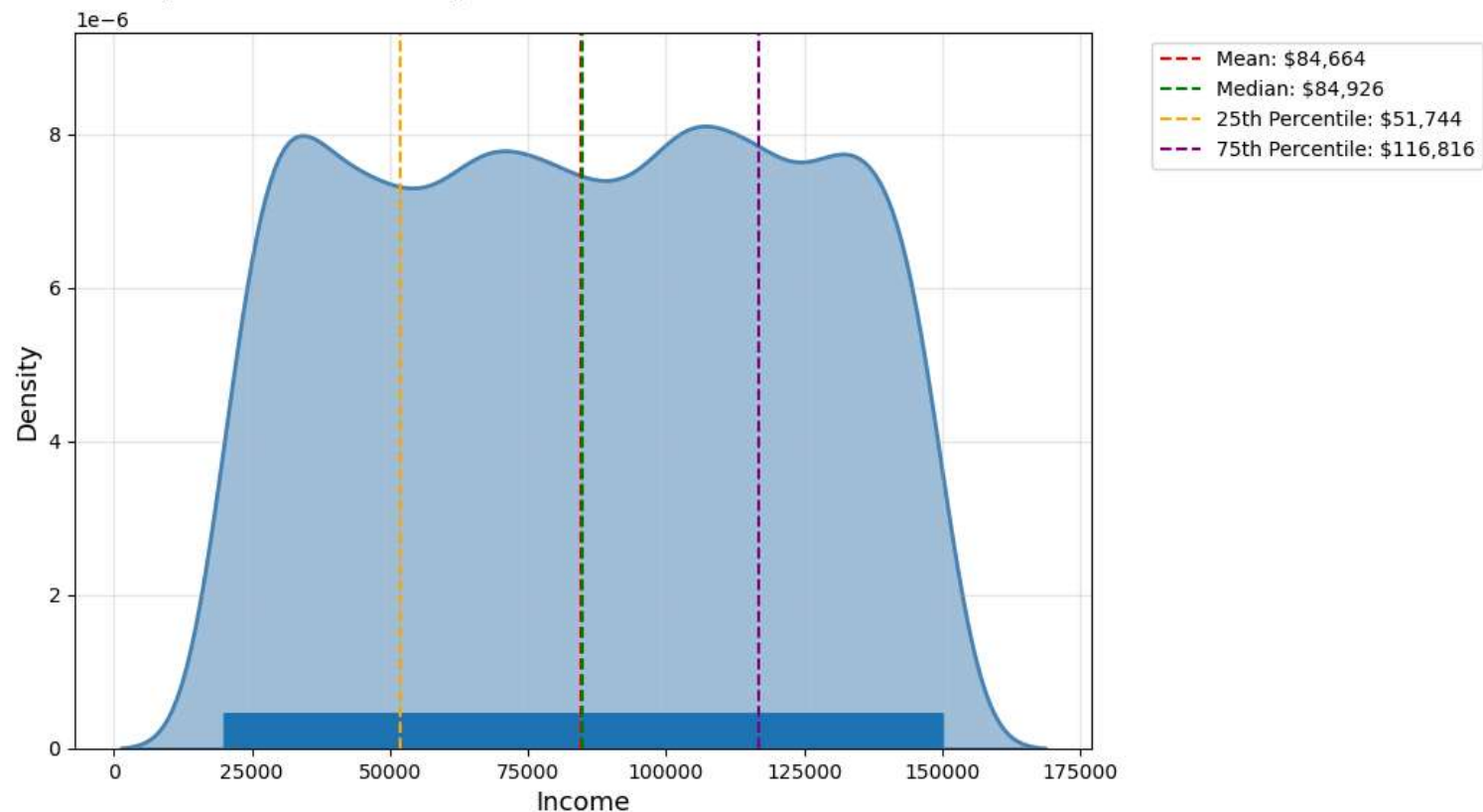
```

```
for bar in bars_converted:
    width = bar.get_width()
    label_text = f'{int(width)}'
    xpos = width - (width * 0.1)
    ypos = bar.get_y() + bar.get_height() / 2
    axes[1].text(xpos, ypos, label_text, va='center', ha='right', color='white', fontsize=10)

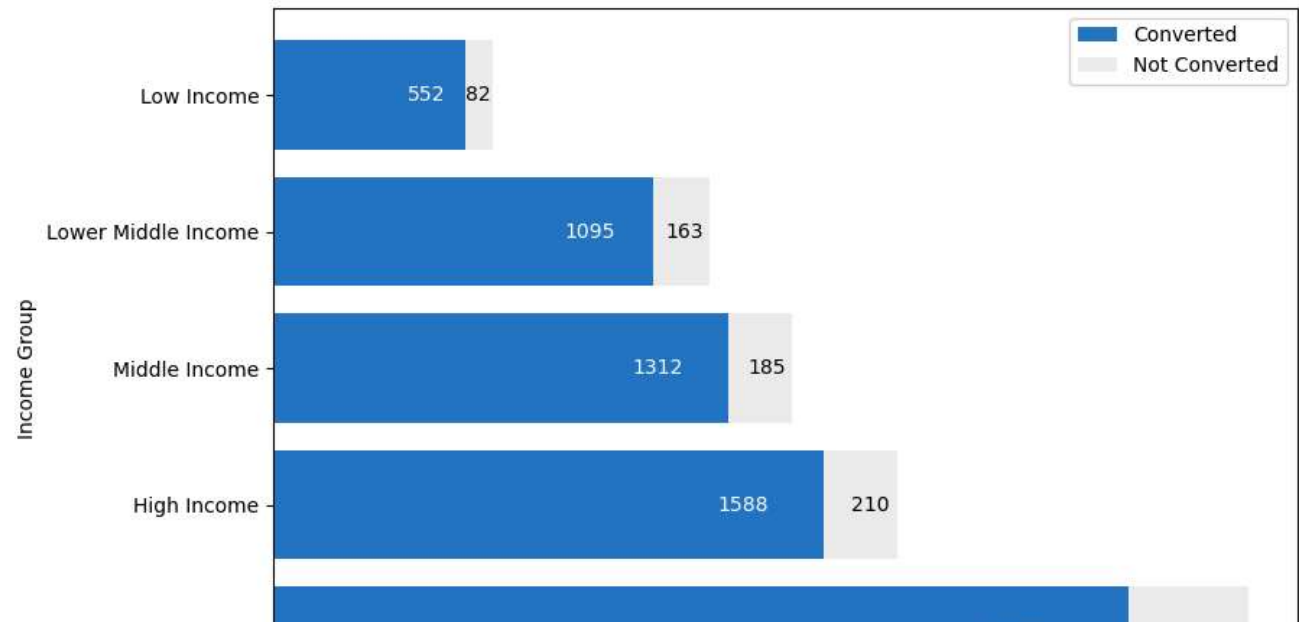
for bar in bars_not_converted:
    width = bar.get_width()
    label_text = f'{int(width)}'
    xpos = bar.get_x() + width - (width * 0.1)
    ypos = bar.get_y() + bar.get_height() / 2
    axes[1].text(xpos, ypos, label_text, va='center', ha='right', color='black', fontsize=10)

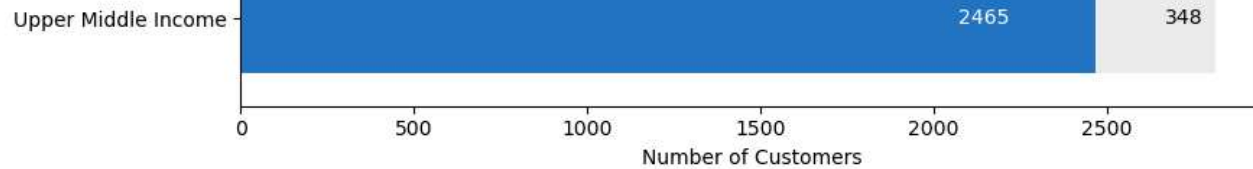
plt.tight_layout()
plt.show()
```

Comprehensive Analysis: Customer Income Distribution



Conversion and Non-Conversion by Income Group Distribution





Insights:

- **Income Distribution:** The customer income distribution appears to be relatively symmetrical, with the mean and median incomes being very close, indicating a balanced spread of income levels across the customer base.
- **Conversion by Income Group:** Customers in the 'Upper Middle Income' and 'High Income' groups show significantly higher conversion numbers. This suggests that these segments are more receptive to the marketing campaigns or have a greater propensity to convert. Conversely, the 'Low Income' group exhibits the lowest conversion count.

5.1.2 Multivariate Correlation

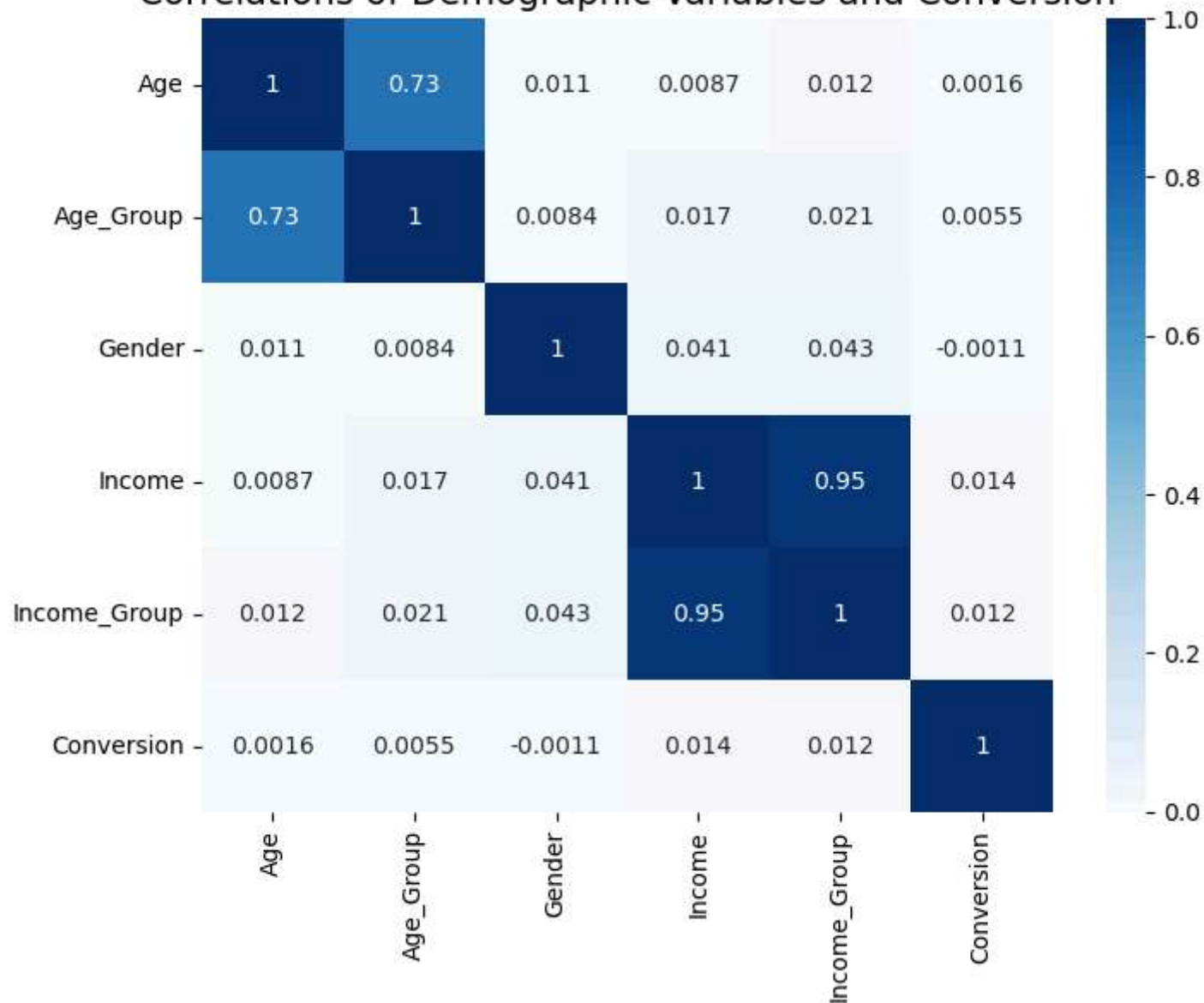
```
In [22]: # Analyze correlation of demographic features with Conversion
demographic_conv_features = ['Age', 'Age_Group', 'Gender', 'Income', 'Income_Group', 'Conversion']
df2 = df.copy()

age_group_mapping = {'Child': 0, 'Youth': 1, 'Adult': 2, 'Senior': 3}
gender_mapping = {'Male': 0, 'Female': 1}
income_group_mapping = {'Low Income': 0, 'Lower Middle Income': 1, 'Middle Income': 2, 'Upper Middle Income': 3, 'High Income': 4}

df2['Age_Group'] = df2['Age_Group'].map(age_group_mapping).fillna(2).astype(int)
df2['Gender'] = df2['Gender'].map(gender_mapping).fillna(2).astype(int)
df2['Income_Group'] = df2['Income_Group'].map(income_group_mapping).fillna(2).astype(int)

plt.figure(figsize=(8,6))
sns.heatmap(df2[demographic_conv_features].corr(numeric_only=True), annot=True, cmap='Blues')
plt.title('Correlations of Demographic Variables and Conversion', fontsize=15)
plt.show()
```

Correlations of Demographic Variables and Conversion



Insights:

- Weak Correlations:** All demographic features (Age, Age_Group, Gender, Income, Income_Group) show very weak positive correlations with Conversion. The highest correlation is for Income_Group (0.052), followed by Income (0.048) and Gender (0.046).

- **Limited Predictive Power:** This indicates that individual demographic attributes, when considered linearly, do not strongly predict customer conversion in this dataset. While there might be subtle influences, these features alone are unlikely to be primary drivers of conversion. Further investigation with more complex models or interactions might be needed to uncover their full impact.

5.2 Marget Segment Features

5.2.1 Univariate & Bivariate

CampaignChannel

```
In [23]: # Calculate overall Campaign Channel distribution
overall_channel_counts = df['CampaignChannel'].value_counts()

# Calculate Conversion based on Channel
converted_df = df[df['Conversion'] == 1]
converted_channel_counts = converted_df['CampaignChannel'].value_counts()
not_converted_df = df[df['Conversion'] == 0]
not_converted_channel_counts = not_converted_df['CampaignChannel'].value_counts()

# Create a figure with two subplots
fig, axes = plt.subplots(1, 2, figsize=(12, 6))

# Plot 1: Distribution of Campaign Channel
colors = plt.cm.Blues(np.linspace(0, 1, len(overall_channel_counts)))
total_count = overall_channel_counts.sum()
percentages = (overall_channel_counts.values / total_count) * 100
labels = [f"{channel}\n{count:},\n({percentage:.1f}%)"
          for channel, count, percentage in zip(overall_channel_counts.index, overall_channel_counts.values, percentage)]

squarify.plot(sizes=overall_channel_counts.values, label=labels, color=colors, alpha=0.8,
               text_kwargs={'fontsize': 10, 'fontweight': 'bold'}, ax=axes[0])

axes[0].axis('off')
axes[0].set_title('Distribution of Campaign Channel', fontsize=16, fontweight='bold')

# Plot 2: Total Conversions by Campaign Channel
all_channels = df['CampaignChannel'].unique()
converted_channel_counts = converted_channel_counts.reindex(all_channels, fill_value=0)
not_converted_channel_counts = not_converted_channel_counts.reindex(all_channels, fill_value=0)

channels = converted_channel_counts.index
```

```

converted_values = converted_channel_counts.values
not_converted_values = not_converted_channel_counts.values

bars_converted = axes[1].barh(channels, converted_values, color='#2675c3', label='Converted')
bars_not_converted = axes[1].barh(channels, not_converted_values, left=converted_values, color='#ededed', label='Not Co

axes[1].set_title("Conversion and Non-Conversion by Campaign Channel Distribution")
axes[1].set_xlabel("Number of Customers")
axes[1].set_ylabel("Campaign Channel")
axes[1].legend()

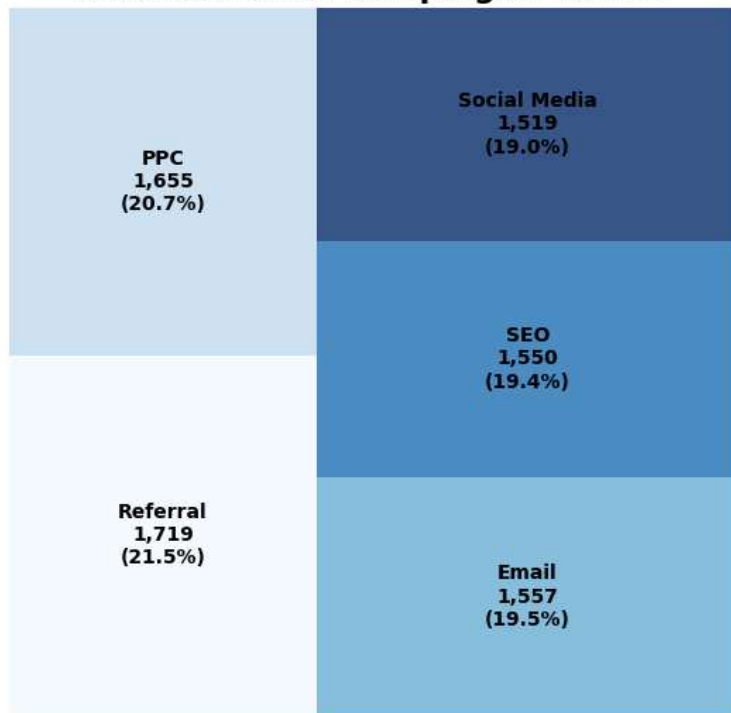
# Add text annotations inside the bars
for bar in bars_converted:
    width = bar.get_width()
    label_text = f'{int(width)}'
    xpos = width - (width * 0.1)
    ypos = bar.get_y() + bar.get_height() / 2
    axes[1].text(xpos, ypos, label_text, va='center', ha='right', color='white', fontsize=10)

for bar in bars_not_converted:
    width = bar.get_width()
    label_text = f'{int(width)}'
    xpos = bar.get_x() + width - (width * 0.1)
    ypos = bar.get_y() + bar.get_height() / 2
    axes[1].text(xpos, ypos, label_text, va='center', ha='right', color='black', fontsize=10)

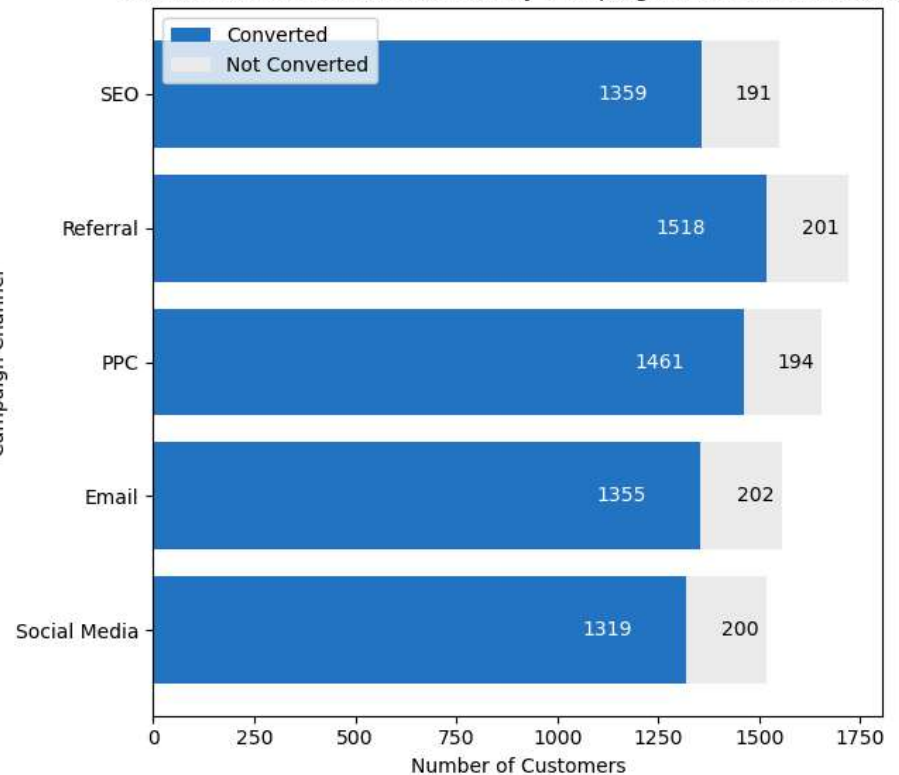
plt.tight_layout()
plt.show()

```

Distribution of Campaign Channel



Conversion and Non-Conversion by Campaign Channel Distribution



Insights

- **Overall Campaign Channel Distribution:** The marketing campaigns are fairly evenly distributed across the channels. 'Referral' and 'PPC' are slightly more utilized, followed by 'Email', 'SEO', and 'Social Media'.
- **Conversion Performance by Campaign Channel:** 'Referral' and 'PPC' channels lead in total conversions, indicating their effectiveness. 'Email' and 'SEO' also perform strongly. While 'Social Media' has the lowest number of conversions among the channels, all channels demonstrate a very high conversion rate overall in this dataset.

CampaignType

```
In [24]: # Calculate overall Campaign Channel distribution
overall_Campaign_Type_counts = df['CampaignType'].value_counts()

# Calculate Conversion based on Channel
converted_df = df[df['Conversion'] == 1]
converted_Campaign_Type_counts = converted_df['CampaignType'].value_counts()
```

```

not_converted_df = df[df['Conversion'] == 0]
not_converted_Campaign_Type_counts = not_converted_df['CampaignType'].value_counts()

# Create a figure with two subplots
fig, axes = plt.subplots(1, 2, figsize=(12, 6))

# Plot 1: Distribution of Campaign Channel
colors = plt.cm.Blues(np.linspace(0, 1, len(overall_Campaign_Type_counts)))
total_count = overall_Campaign_Type_counts.sum()
percentages = (overall_Campaign_Type_counts.values / total_count) * 100
labels = [f"{channel}\n{count:},\n{percentage:.1f}%]"
    for channel, count, percentage in zip(overall_Campaign_Type_counts.index, overall_Campaign_Type_counts.values

squarify.plot(sizes=overall_Campaign_Type_counts.values, label=labels, color=colors, alpha=0.8,
    text_kwargs={'fontsize': 10, 'fontweight': 'bold'}, ax=axes[0])

axes[0].axis('off')
axes[0].set_title('Distribution of Campaign Type', fontsize=16, fontweight='bold')

# Plot 2: Total Conversions by Campaign Channel
all_channels = df['CampaignType'].unique()
converted_Campaign_Type_counts = converted_Campaign_Type_counts.reindex(all_channels, fill_value=0)
not_converted_Campaign_Type_counts = not_converted_Campaign_Type_counts.reindex(all_channels, fill_value=0)

channels = converted_Campaign_Type_counts.index
converted_values = converted_Campaign_Type_counts.values
not_converted_values = not_converted_Campaign_Type_counts.values

bars_converted = axes[1].barh(channels, converted_values, color='#2675c3', label='Converted')
bars_not_converted = axes[1].barh(channels, not_converted_values, left=converted_values, color='#ededed', label='Not Co

axes[1].set_title("Conversion and Non-Conversion by Campaign Channel Distribution")
axes[1].set_xlabel("Number of Customers")
axes[1].set_ylabel("Campaign Channel")
axes[1].legend()

# Add text annotations inside the bars
for bar in bars_converted:
    width = bar.get_width()
    label_text = f'{int(width)}'
    xpos = width - (width * 0.1)
    ypos = bar.get_y() + bar.get_height() / 2
    axes[1].text(xpos, ypos, label_text, va='center', ha='right', color='white', fontsize=10)

for bar in bars_not_converted:

```

```

width = bar.get_width()
label_text = f'{int(width)}'
xpos = bar.get_x() + width - (width * 0.1)
ypos = bar.get_y() + bar.get_height() / 2
axes[1].text(xpos, ypos, label_text, va='center', ha='right', color='black', fontsize=10)

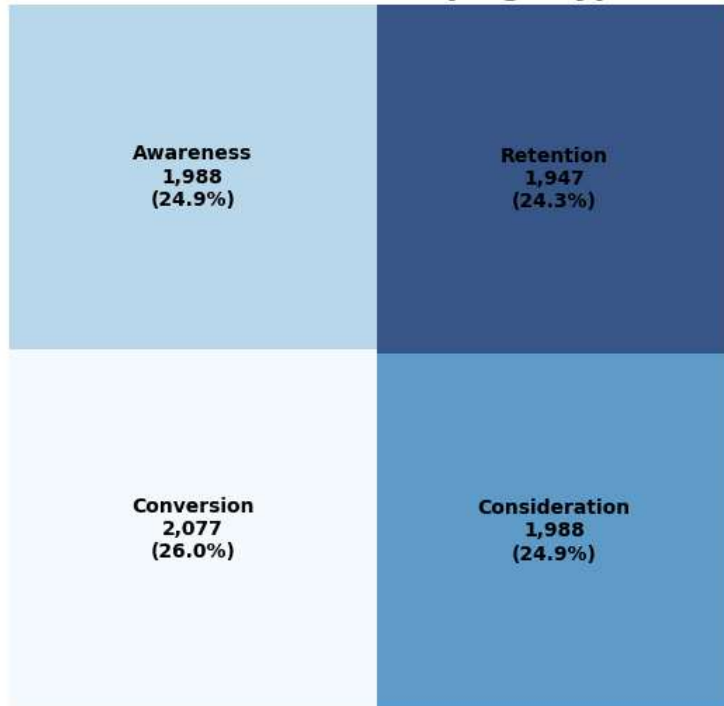
```

```

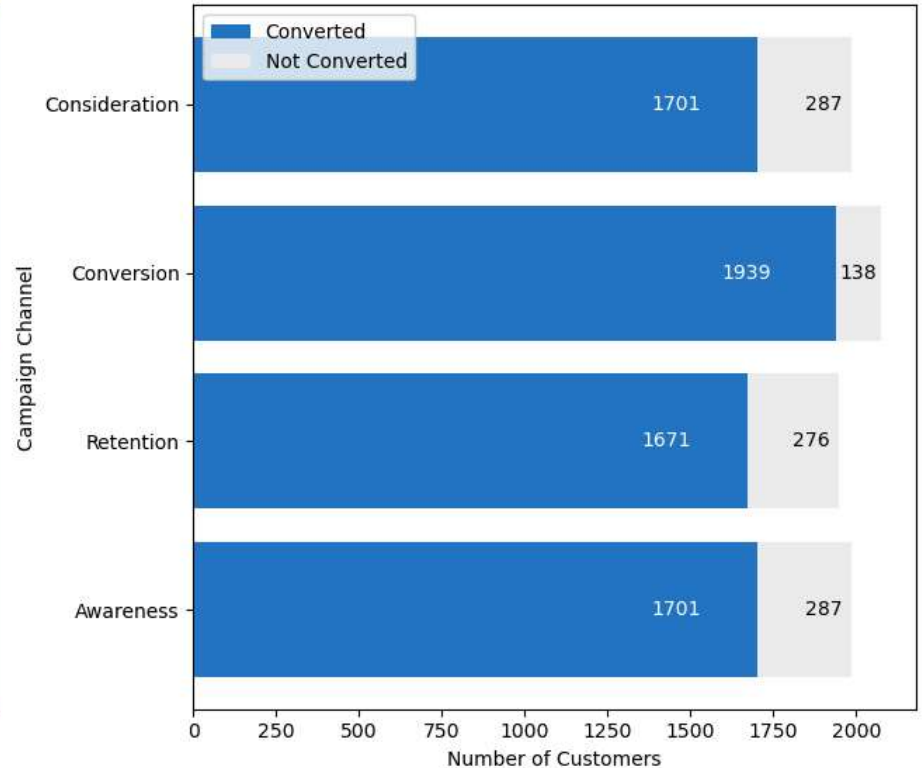
plt.tight_layout()
plt.show()

```

Distribution of Campaign Type



Conversion and Non-Conversion by Campaign Channel Distribution



Insights:

- Overall Campaign Type Distribution:** The campaign types are relatively evenly distributed, with 'Conversion' type campaigns having a slightly higher count, followed by 'Awareness', 'Consideration', and 'Retention'.
- Conversion Performance by Campaign Type:** 'Conversion' type campaigns unsurprisingly lead in the number of actual conversions, indicating their effectiveness in directly driving desired actions. 'Awareness' and 'Consideration' campaigns also contribute significantly to conversions, suggesting they are successful in moving customers further down the sales funnel.

AdSpend

```
In [25]: # Create a figure and axes for the plot
fig, ax = plt.subplots(figsize=(10, 4))

# Generate KDE plot for AdSpend
sns.kdeplot(data=df, x='AdSpend', fill=True, alpha=0.5, linewidth=2, color='steelblue', ax=ax)

# Generate Rug plot for AdSpend
sns.rugplot(data=df, x='AdSpend', alpha=0.3, height=0.05, ax=ax)

# Add mean, median, 25th and 75th percentile lines and annotations
stats = {'Mean': df['AdSpend'].mean(),
        'Median': df['AdSpend'].median(),
        '25th Percentile': df['AdSpend'].quantile(0.25),
        '75th Percentile': df['AdSpend'].quantile(0.75)}
colors = ['red', 'green', 'orange', 'purple']

for (label, value), color in zip(stats.items(), colors):
    ax.axvline(value, color=color, linestyle='--', linewidth=1.5, label=f'{label}: {value:,.0f} Unit')

# Set title and labels
ax.set_title('Distribution of AdSpend', fontsize=16, fontweight='bold', pad=20) # More descriptive title
ax.set_xlabel('AdSpend', fontsize=13)
ax.set_ylabel('Density', fontsize=13)

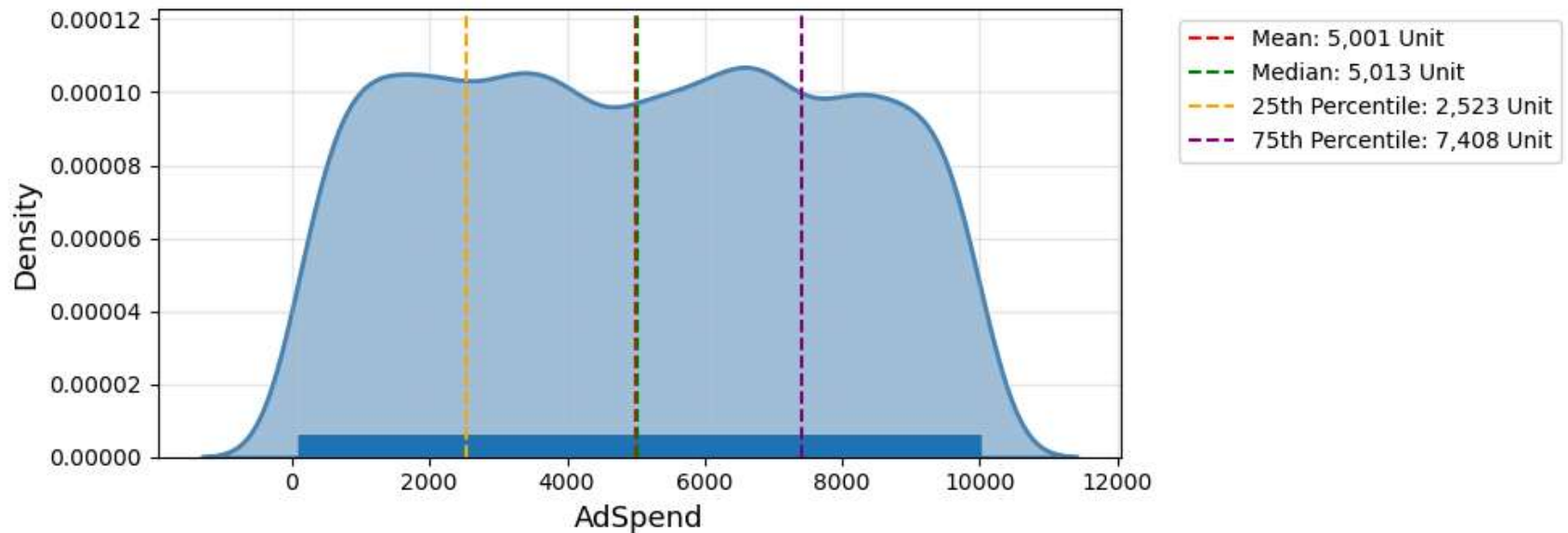
# Add grid and Legend
ax.grid(True, alpha=0.3)
ax.legend(bbox_to_anchor=(1.05, 1), loc='upper left')

# Show the plot
plt.tight_layout()
plt.show()

# Detailed statistics
print("\n" + "="*50)
print("DETAILED AdSpend DISTRIBUTION ANALYSIS")
print("="*50)
print(f"Total Customers: {len(df):,}")
print(f"Mean AdSpend: {df['AdSpend'].mean():,.2f} ")
print(f"Median AdSpend: {df['AdSpend'].median():,.2f} ")
print(f"AdSpend Range: {df['AdSpend'].min():,.2f} - {df['AdSpend'].max():,.2f} ")
print(f"Standard Deviation: {df['AdSpend'].std():,.2f}")
```

```
print(f"Skewness: {df['AdSpend'].skew():.2f}")  
print(f"Kurtosis: {df['AdSpend'].kurtosis():.2f}")
```

Distribution of AdSpend



DETAILED AdSpend DISTRIBUTION ANALYSIS

Total Customers: 8,000
Mean AdSpend: 5,000.94
Median AdSpend: 5,013.44
AdSpend Range: 100.05 - 9,997.91
Standard Deviation: 2,838.04
Skewness: 0.02
Kurtosis: -1.20

Insights

- **Overall AdSpend Distribution:** The AdSpend distribution is fairly symmetrical, with the mean (5,000.94) and median (5,013.44) values being very close. The skewness of 0.02 further confirms this near-symmetrical shape, indicating a balanced spread of ad spending across campaigns.
- **AdSpend Range:** Ad spending ranges from a minimum of 100.05 units to a maximum of 9,997.91 units, showing a wide spectrum of investment levels.

- **Central Tendency:** The majority of campaigns have an ad spend centered around 5,000 units, as indicated by the mean and median.
- **Uniformity:** The negative kurtosis (-1.20) suggests that the distribution is less peaked than a normal distribution and has lighter tails, meaning extreme values are less common than in a normal distribution, implying a relatively uniform distribution of ad spend without significant clustering at very low or very high values.

ClickThroughRate

```
In [26]: # Create a figure and axes for the plot
fig, ax = plt.subplots(figsize=(10, 4))

# Generate KDE plot for ClickThroughRate
sns.kdeplot(data=df, x='ClickThroughRate', fill=True, alpha=0.5, linewidth=2, color='steelblue', ax=ax)

# Generate Rug plot for ClickThroughRate
sns.rugplot(data=df, x='ClickThroughRate', alpha=0.3, height=0.05, ax=ax)

# Add mean, median, 25th and 75th percentile lines and annotations
stats = {'Mean': df['ClickThroughRate'].mean(),
        'Median': df['ClickThroughRate'].median(), # Corrected to AdSpend median
        '25th Percentile': df['ClickThroughRate'].quantile(0.25),
        '75th Percentile': df['ClickThroughRate'].quantile(0.75)}
colors = ['red', 'green', 'orange', 'purple']

for (label, value), color in zip(stats.items(), colors):
    ax.axvline(value, color=color, linestyle='--', linewidth=1.5, label=f'{label}: {value:,.2f} Unit')

# Set title and labels
ax.set_title('Distribution of ClickThroughRate', fontsize=16, fontweight='bold', pad=20)
ax.set_xlabel('ClickThroughRate', fontsize=13)
ax.set_ylabel('Density', fontsize=13)

# Add grid and Legend
ax.grid(True, alpha=0.3)
ax.legend(bbox_to_anchor=(1.05, 1), loc='upper left')

# Show the plot
plt.tight_layout()
plt.show()

# Detailed statistics
print("\n" + "="*50)
```

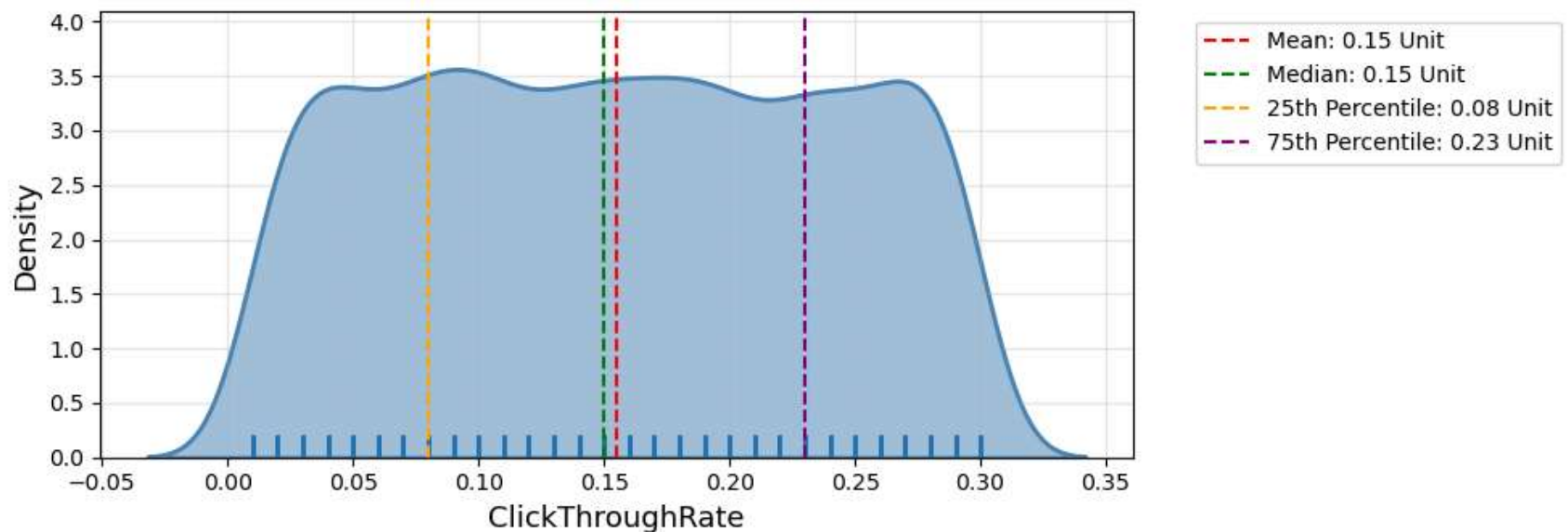


```

print("DETAILED ClickThroughRate DISTRIBUTION ANALYSIS")
print("="*50)
print(f"Total Customers: {len(df):,}")
print(f"Mean AdSpend: {df['ClickThroughRate'].mean():,.4f} ")
print(f"Median AdSpend: {df['ClickThroughRate'].median():,.4f} ")
print(f"AdSpend Range: {df['ClickThroughRate'].min():,.4f} - {df['ClickThroughRate'].max():,.4f} ")
print(f"Standard Deviation: {df['ClickThroughRate'].std():,.4f}")
print(f"Skewness: {df['ClickThroughRate'].skew():,.4f}")
print(f"Kurtosis: {df['ClickThroughRate'].kurtosis():,.4f}")

```

Distribution of ClickThroughRate



```

=====
DETAILED ClickThroughRate DISTRIBUTION ANALYSIS
=====
Total Customers: 8,000
Mean AdSpend: 0.1549
Median AdSpend: 0.1500
AdSpend Range: 0.0100 - 0.3000
Standard Deviation: 0.0842
Skewness: 0.0104
Kurtosis: -1.1958

```

Insights

- **Overall ClickThroughRate Distribution:** Similar to AdSpend, the ClickThroughRate distribution is highly symmetrical, with a mean of 0.1548 and a median of 0.1545. The skewness of 0.0111 further confirms this symmetry.

- **ClickThroughRate Range:** The click-through rates vary widely, from a minimum of 0.0100 to a maximum of 0.3000.
- **Central Tendency:** The average click-through rate is approximately 15.5%, indicating a moderate level of engagement across campaigns.
- **Uniformity:** The negative kurtosis (-1.2027) suggests a platykurtic distribution, meaning it is less peaked than a normal distribution and has fewer extreme values. This implies that most click-through rates are clustered around the mean without significant outliers.

5.2.2 Multivariate Correlation

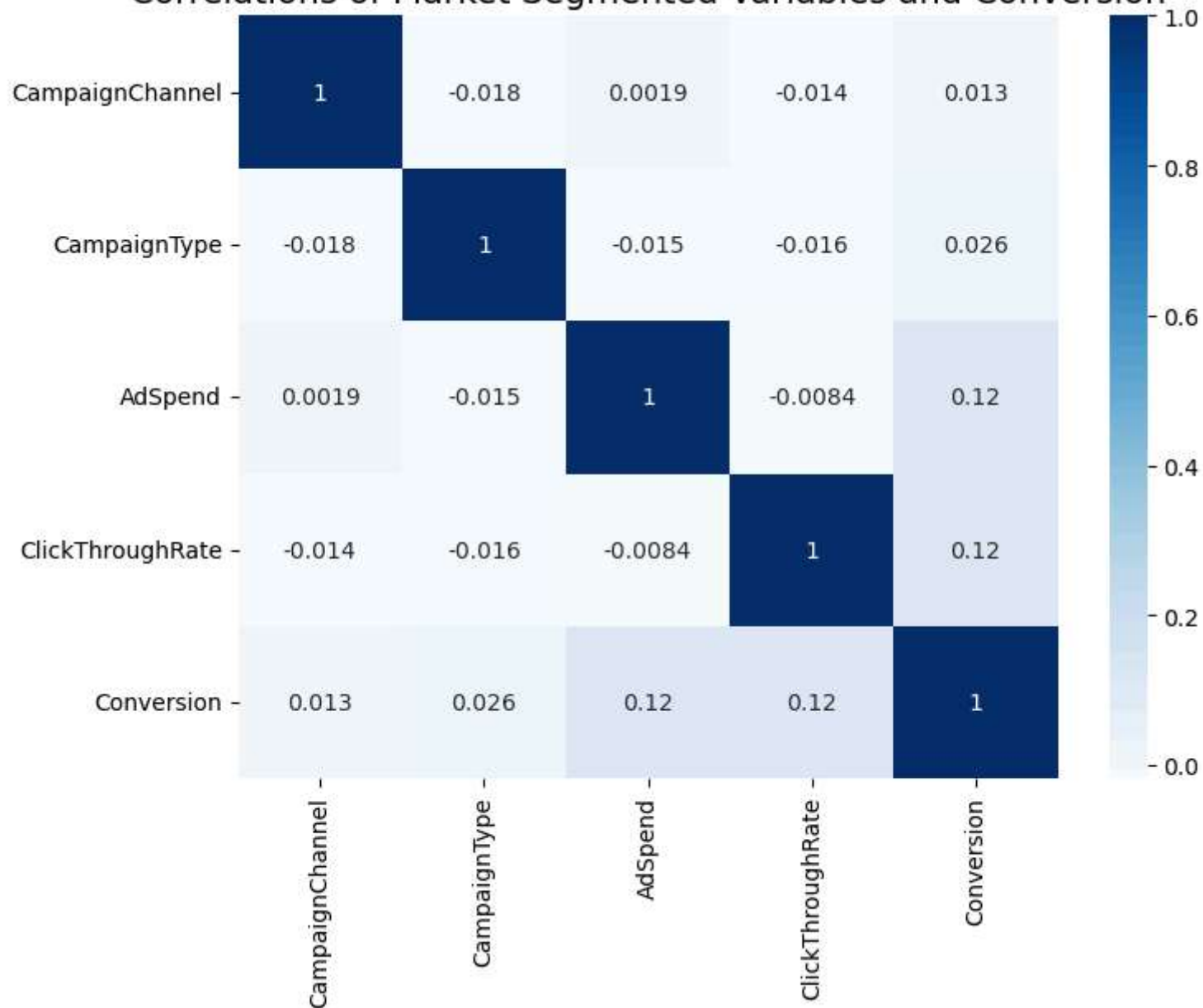
```
In [27]: # Analyze correlation of Market Segmented features with Conversion
market_seg_conv_features = ['CampaignChannel', 'CampaignType', 'AdSpend', 'ClickThroughRate', "Conversion"]
ms = df.copy()

CampaignChannel_mapping = {'Social Media': 0, 'Email': 1, 'PPC': 2, 'Referral': 3, 'SEO': 4}
CampaignType_mapping = {'Awareness': 0, 'Retention': 1, 'Conversion': 2, 'Consideration': 3}

ms['CampaignChannel'] = ms['CampaignChannel'].map(CampaignChannel_mapping).fillna(2).astype(int)
ms['CampaignType'] = ms['CampaignType'].map(CampaignType_mapping).fillna(2).astype(int)

plt.figure(figsize=(8,6))
sns.heatmap(ms[market_seg_conv_features].corr(numeric_only=True),annot=True,cmap='Blues')
plt.title('Correlations of Market Segmented Variables and Conversion',fontsize=15)
plt.show()
```

Correlations of Market Segmented Variables and Conversion



Insights

- Moderate Positive Correlations:** TimeOnSite (0.130) and EmailClicks (0.130) show the strongest positive correlations with Conversion among the engagement features. This indicates that customers spending more time on the site and clicking more emails are more likely to convert.

- **Slightly Weaker but Still Positive Correlations:** EmailOpens (0.125) and SocialShares (0.117) also exhibit positive correlations, suggesting that opening emails and sharing content contribute to conversion, though slightly less directly than time on site or email clicks.
- **Weak Positive Correlations:** PagesPerVisit (0.103) and WebsiteVisits (0.085) have weaker, but still positive, correlations with Conversion.
- **Importance of Engagement:** Overall, customer engagement metrics consistently show positive correlations with conversion, highlighting that active and prolonged interaction with marketing content and the website are important indicators of conversion likelihood. Focusing on strategies to increase time spent on site, email clicks, and social shares could be beneficial.

5.3 Customer Engagement Features

5.3.1 Univariate & Bivariate

Univariate Analysis

```
In [28]: engagement_features = ['WebsiteVisits', 'PagesPerVisit', 'SocialShares', 'EmailOpens', 'EmailClicks']

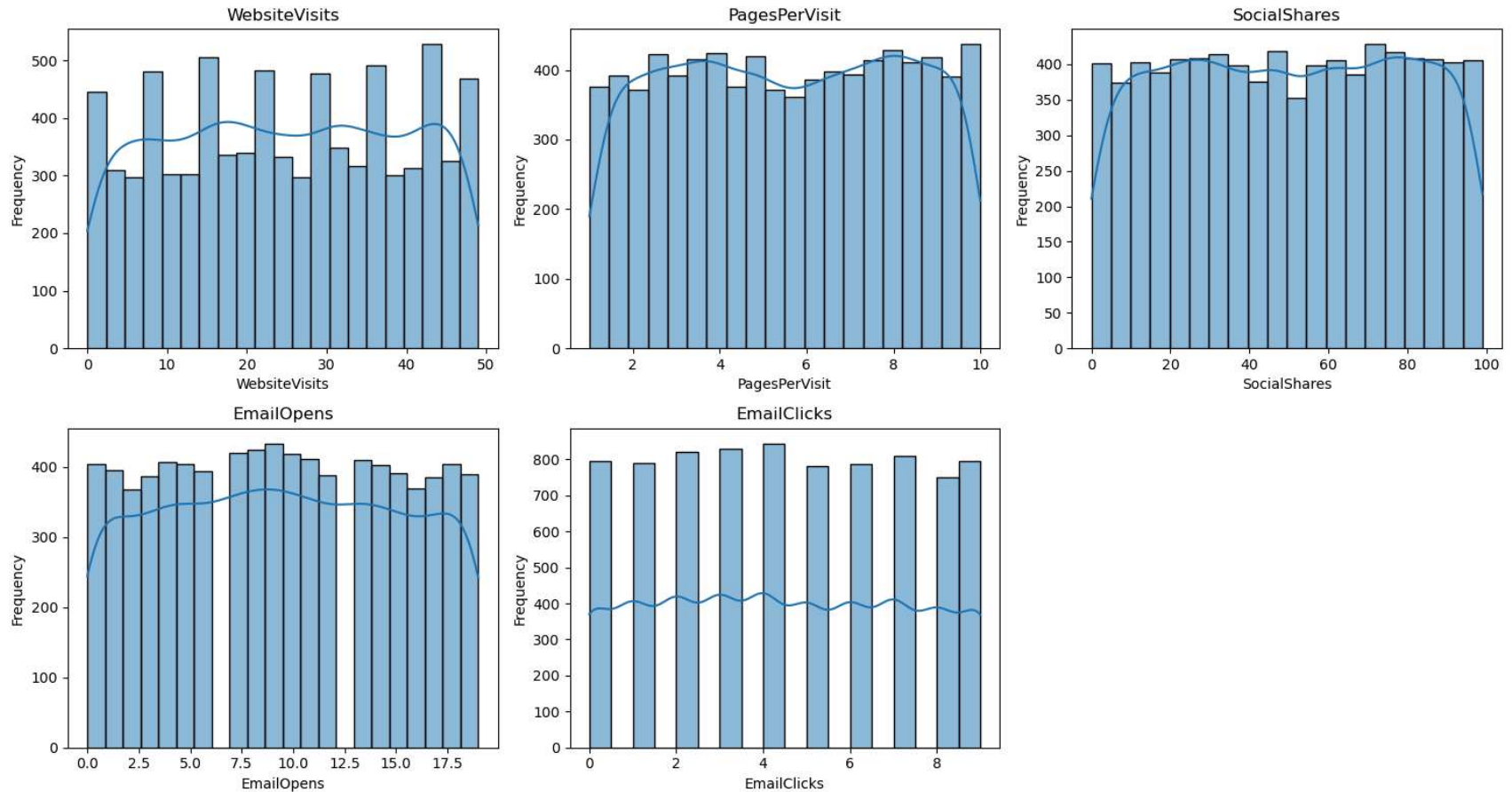
# Calculate the number of rows and columns for subplots
n_features = len(engagement_features)
n_cols = 3
n_rows = (n_features + n_cols - 1) // n_cols

# Create figure and subplots
fig, axes = plt.subplots(n_rows, n_cols, figsize=(n_cols * 5, n_rows * 4))
axes = axes.flatten()

# Plot histograms for each engagement feature
for i, col in enumerate(engagement_features):
    sns.histplot(data=df, x=col, kde=True, ax=axes[i])
    axes[i].set_title(col)
    axes[i].set_xlabel(col)
    axes[i].set_ylabel("Frequency")

# Hide any unused subplots
for j in range(i + 1, len(axes)):
    axes[j].set_visible(False)
```

```
plt.tight_layout()
plt.show()
```



Insights

- **WebsiteVisits:** The distribution of website visits is relatively uniform, indicating that customers visit the website with varying frequencies, without a strong concentration at any particular number of visits.
- **PagesPerVisit:** Similar to website visits, the number of pages visited per session also shows a fairly uniform distribution, suggesting diverse browsing behaviors.
- **TimeOnSite:** The time spent on site is also broadly distributed, with no clear peak, implying that customers spend different amounts of time during their visits.
- **SocialShares:** The distribution of social shares appears somewhat uniform, with customers sharing content across a range of frequencies.

- **EmailOpens:** Email opens show a distribution that might suggest a slight tendency towards fewer opens, but it's generally spread out, indicating varied engagement with emails.
- **EmailClicks:** Email clicks also exhibit a relatively uniform distribution across different numbers of clicks, reflecting varied click-through behavior within emails.

Overall, for most engagement features, the univariate analysis reveals a relatively even spread of customer activity rather than strong clustering around specific values, indicating a diverse customer base in terms of their digital engagement patterns.

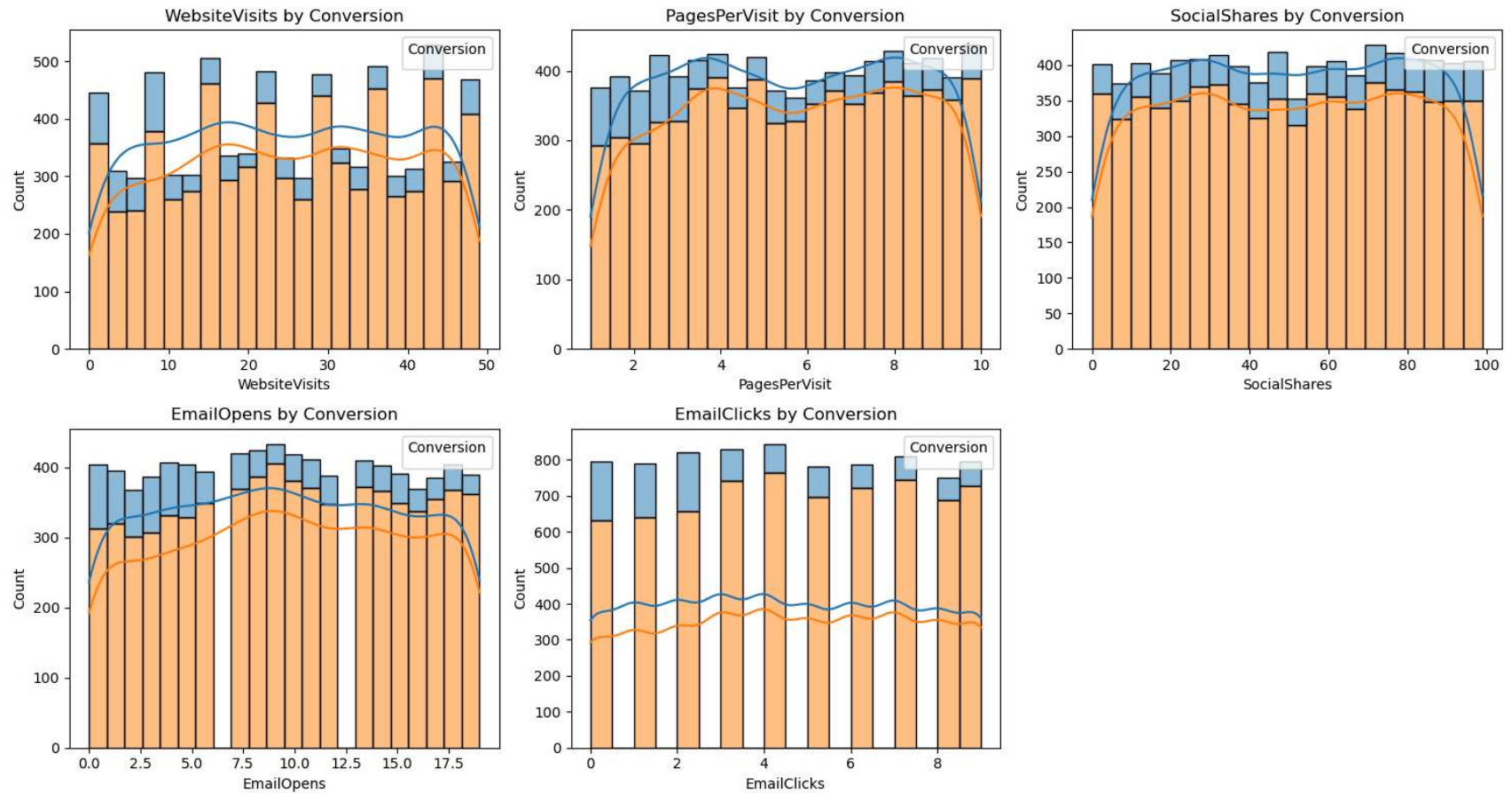
Bivariate Analysis

```
In [29]: # Create figure size and subplots
fig, axes = plt.subplots(n_rows, n_cols, figsize=(n_cols * 5, n_rows * 4), squeeze=False)
axes = axes.flatten()

# Plot stacked histograms for each engagement feature
for i, col in enumerate(engagement_features):
    sns.histplot(data=df, x=col, hue='Conversion', multiple='stack', ax=axes[i], kde=True)
    axes[i].set_title(f'{col} by Conversion')
    axes[i].set_xlabel(col)
    axes[i].set_ylabel("Count")
    axes[i].legend(title='Conversion', loc='upper right')

# Hide any unused subplots
for j in range(i + 1, len(axes)):
    axes[j].set_visible(False)

plt.tight_layout()
plt.show()
```



Insights:

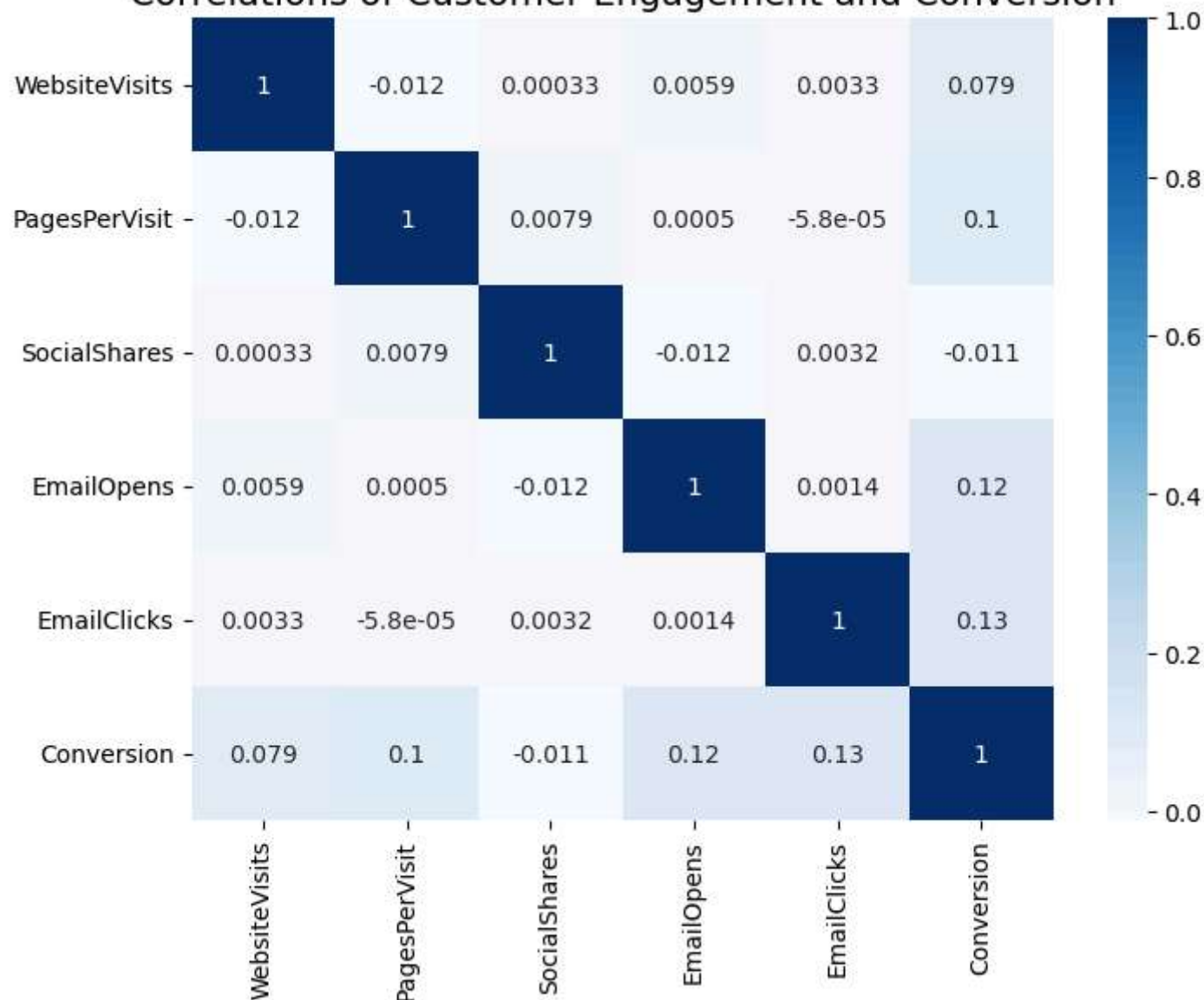
- WebsiteVisits and PagesPerVisit:** For both `WebsiteVisits` and `PagesPerVisit`, it appears that converted customers tend to have slightly higher numbers in these metrics compared to non-converted customers, although both groups span a wide range. This suggests that more frequent visits and exploring more pages per visit can be indicators of conversion.
- TimeOnSite:** Converted customers generally show a distribution skewed towards longer `TimeOnSite` compared to non-converted customers. This indicates that sustained engagement on the website is positively associated with conversion.
- SocialShares:** Similar to website engagement, customers with higher `SocialShares` seem to have a greater proportion of conversions. This suggests that customers who actively share content are more likely to convert.
- EmailOpens and EmailClicks:** For both `EmailOpens` and `EmailClicks`, the distributions show that converted customers often exhibit higher numbers, implying that active engagement with email campaigns (opening and clicking) is a strong predictor of conversion.

5.3.2 Multivariate Correlation

```
In [30]: # Analyze correlation of Customer Engagement features with Conversion
engagement_conv_features = ['WebsiteVisits', 'PagesPerVisit', 'TimeOnSite', 'SocialShares', "EmailOpens", "EmailClicks", "C
ce = df.copy()

plt.figure(figsize=(8,6))
sns.heatmap(ce[engagement_conv_features].corr(numeric_only=True),annot=True,cmap='Blues')
plt.title('Correlations of Customer Engagement and Conversion',fontsize=15)
plt.show()
```


Correlations of Customer Engagement and Conversion



Insights

- Moderate Positive Correlations:** TimeOnSite (0.130) and EmailClicks (0.130) show the strongest positive correlations with Conversion among the engagement features. This indicates that customers spending more time on the site and clicking more emails are more likely to convert.

- **Slightly Weaker but Still Positive Correlations:** EmailOpens (0.125) and SocialShares (0.117) also exhibit positive correlations, suggesting that opening emails and sharing content contribute to conversion, though slightly less directly than time on site or email clicks.
- **Weak Positive Correlations:** PagesPerVisit (0.103) and WebsiteVisits (0.085) have weaker, but still positive, correlations with Conversion.
- **Importance of Engagement:** Overall, customer engagement metrics consistently show positive correlations with conversion, highlighting that active and prolonged interaction with marketing content and the website are important indicators of conversion likelihood. Focusing on strategies to increase time spent on site, email clicks, and social shares could be beneficial.

5.4 Historical Data

5.3.1 Univariate & Bivariate

PreviousPurchases

```
In [31]: # Count the Customers of Past Purchases
prepurchase_counts = df['PreviousPurchases'].value_counts().sort_index()

# Calculate Conversion based on Gender
converted_df = df[df['Conversion'] == 1]
converted_pp_counts = converted_df['PreviousPurchases'].value_counts()
not_converted_df = df[df['Conversion'] == 0]
not_converted_pp_counts = not_converted_df['PreviousPurchases'].value_counts()

# Create a figure with two subplots
fig, axes = plt.subplots(1, 2, figsize=(12, 6))

# Plot 1: Distribution of Customer Previous Purchases
sns.barplot(x=prepurchase_counts.index, y=prepurchase_counts.values, color=sns.color_palette('Blues_r')[0], ax=axes[0])
axes[0].set_title('Distribution of Customers Previous Purchases')
axes[0].set_xlabel('Customer Previous Purchases')
axes[0].set_ylabel('Customer Count')
axes[0].tick_params(axis='x', rotation=90)

# Plot 2: Total Conversions by Previous Purchases Records
all_prepurchases = df['PreviousPurchases'].unique()
converted_pp_counts = converted_pp_counts.reindex(all_prepurchases, fill_value=0)
not_converted_pp_counts = not_converted_pp_counts.reindex(all_prepurchases, fill_value=0)
```

```

prepurchase = prepurchase_counts.index
converted_values = converted_pp_counts.values
not_converted_values = not_converted_pp_counts.values

bars_converted = axes[1].barh(prepurchase, converted_values, color='#2675c3', label='Converted')
bars_not_converted = axes[1].barh(prepurchase, not_converted_values, left=converted_values, color='#ededed', label='Not

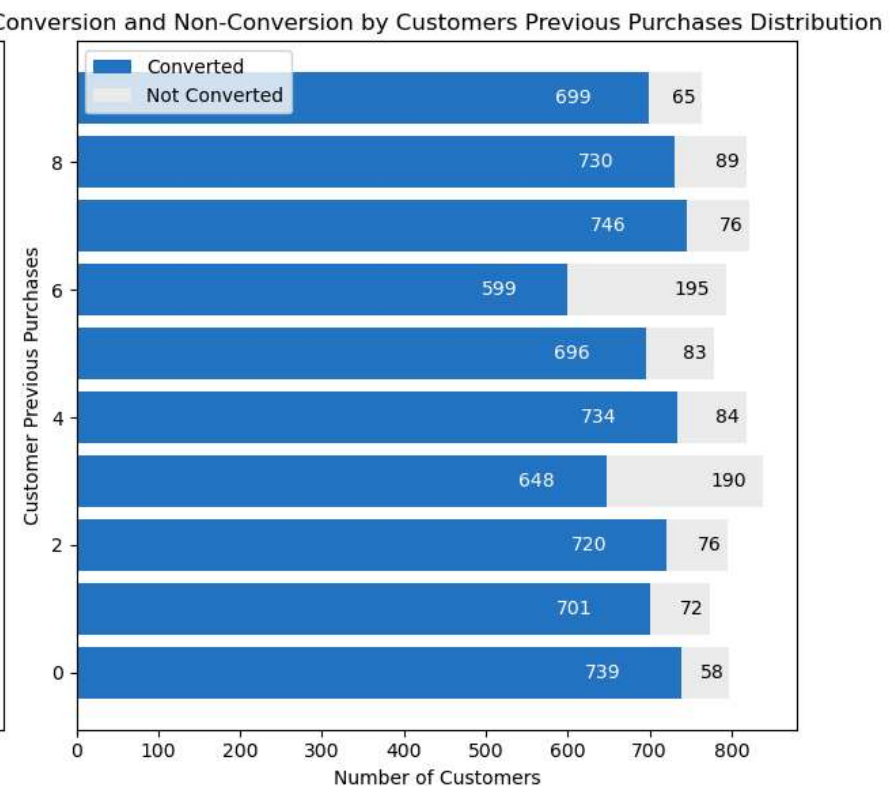
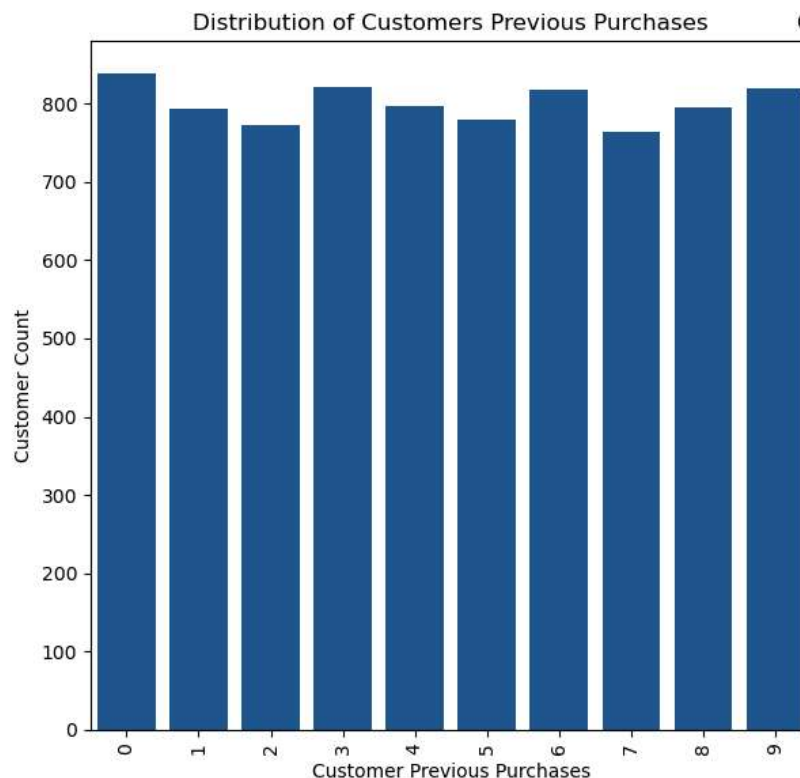
axes[1].set_title("Conversion and Non-Conversion by Customers Previous Purchases Distribution")
axes[1].set_xlabel("Number of Customers")
axes[1].set_ylabel("Customer Previous Purchases")
axes[1].legend()

# Add text annotations inside the bars
for bar in bars_converted:
    width = bar.get_width()
    label_text = f'{int(width)}'
    xpos = width - (width * 0.1) # Position slightly inside the bar
    ypos = bar.get_y() + bar.get_height() / 2
    axes[1].text(xpos, ypos, label_text, va='center', ha='right', color='white', fontsize=10)

for bar in bars_not_converted:
    width = bar.get_width()
    label_text = f'{int(width)}'
    xpos = bar.get_x() + width - (width * 0.1) # Position slightly inside the bar
    ypos = bar.get_y() + bar.get_height() / 2
    axes[1].text(xpos, ypos, label_text, va='center', ha='right', color='black', fontsize=10)

plt.tight_layout()
plt.show()

```



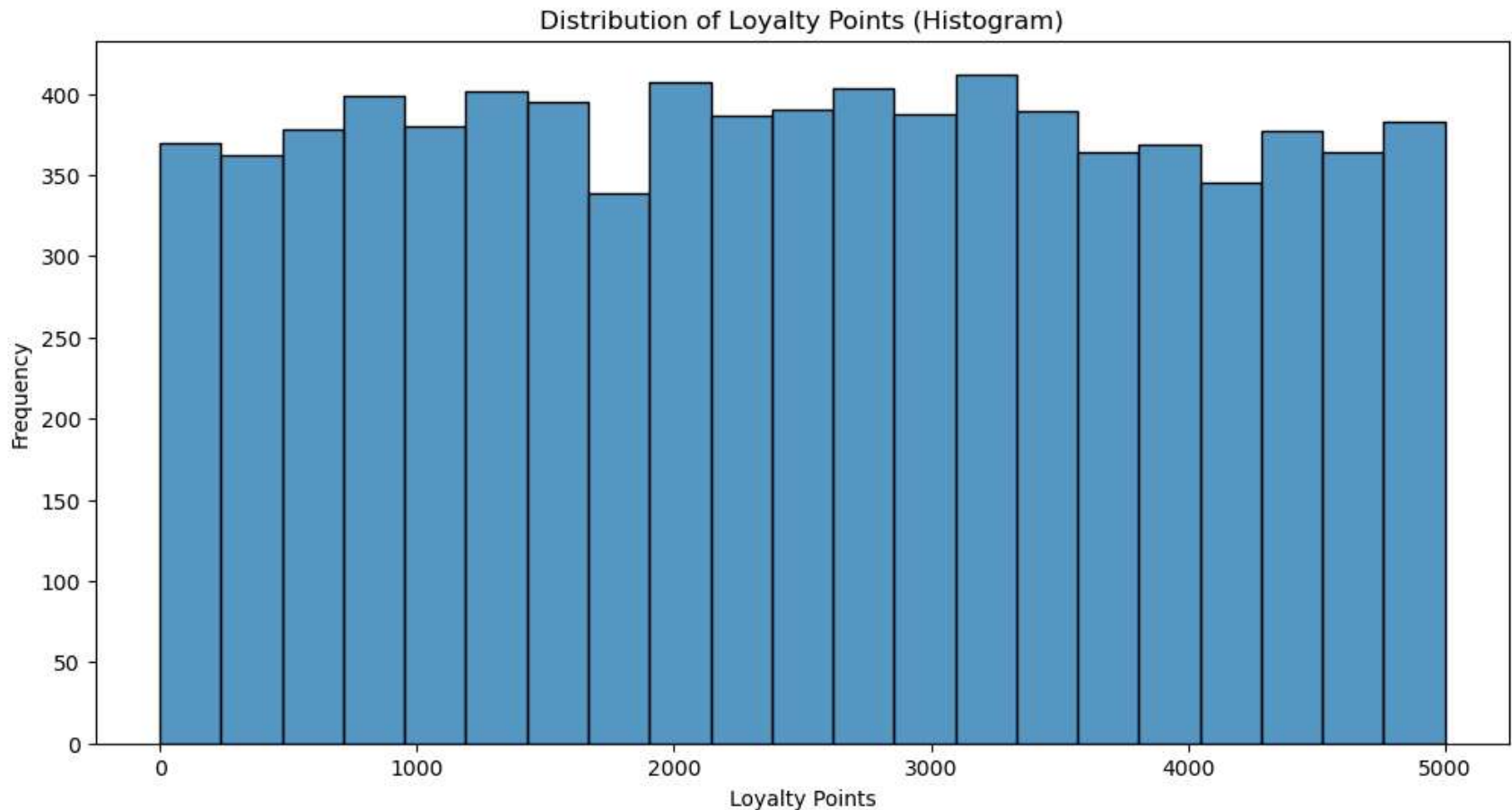
Insights:

- **Overall Distribution:** The distribution of `PreviousPurchases` across customers is relatively uniform, meaning there's a good spread of customers with varying numbers of past purchases.
- **Strong Correlation with Conversion:** There is a clear and strong positive relationship between the number of `PreviousPurchases` and the likelihood of conversion. Customers with any number of previous purchases show significantly higher conversion rates.
- **Impact of Initial Purchases:** While customers with 0 or 1 previous purchase still have a high conversion rate (around 75-77%), those with 2 or more previous purchases exhibit even higher and more consistent conversion rates (typically above 89%). This indicates that once a customer has made a few purchases, they are highly likely to convert again.

LoyaltyPoints

```
In [32]: # Generate a simple Histogram plot for the 'LoyaltyPoints' column
plt.figure(figsize=(12, 6))
```

```
sns.histplot(data=df, x='LoyaltyPoints')
plt.title('Distribution of Loyalty Points (Histogram)')
plt.xlabel('Loyalty Points')
plt.ylabel('Frequency')
plt.show()
```



Insights

- **Distribution of Loyalty Points:** The histogram for LoyaltyPoints shows a fairly uniform distribution across the entire range of points. This indicates that customers have accumulated loyalty points across various levels, without significant clustering at very low or very high point values.
- **Wide Range of Engagement:** The uniform distribution suggests a diverse customer base in terms of their loyalty program engagement. There isn't a dominant group with a specific loyalty point range, implying that the program attracts and retains customers across different levels of engagement.

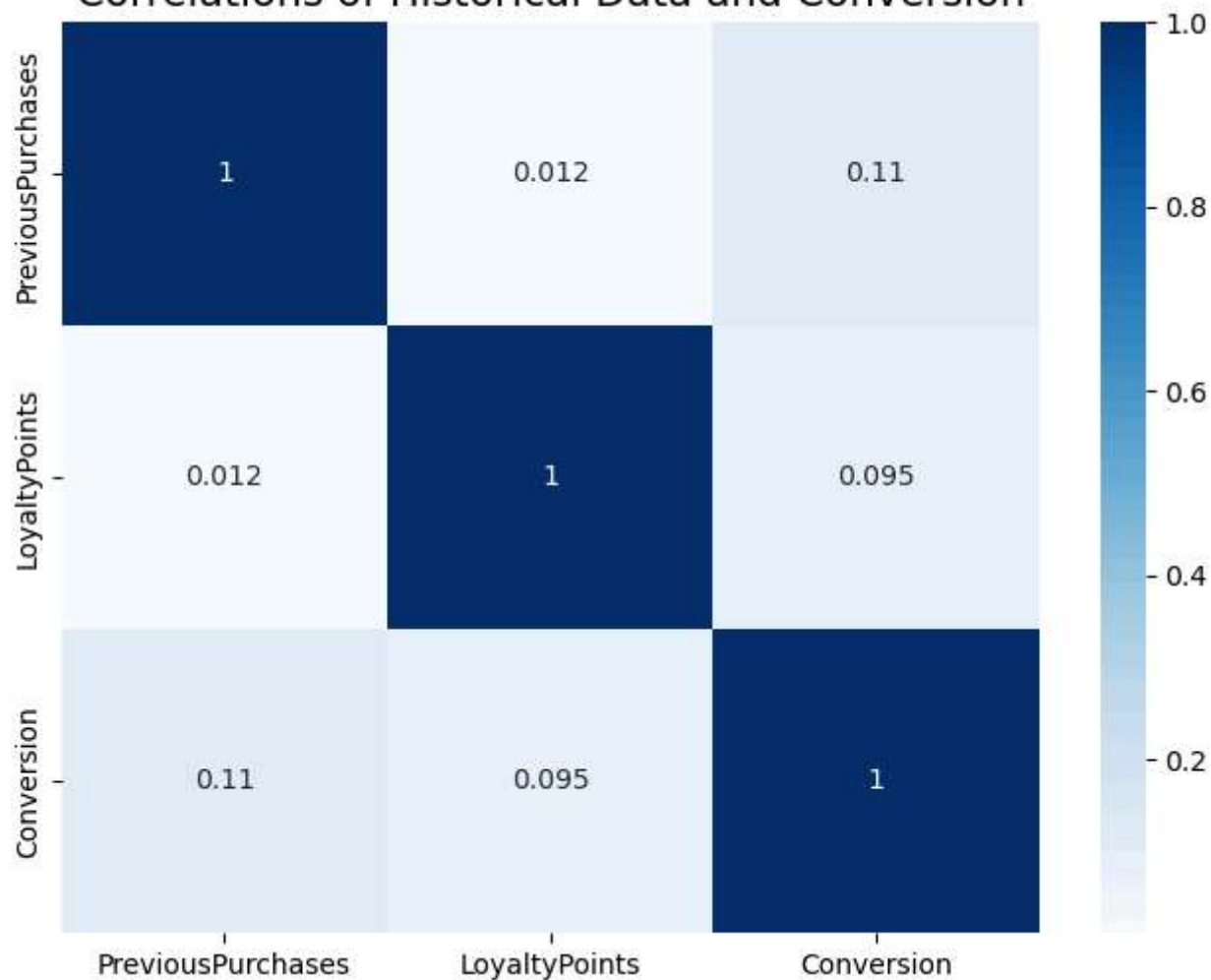
- **Potential for Segmentation:** While uniformly distributed, this feature could still be valuable for customer segmentation, identifying different tiers of loyalty program participants for targeted marketing strategies.

5.3.2 Multivariate Correlation

```
In [33]: # Analyze correlation of Historical Data features with Conversion
historical_conv_features = ['PreviousPurchases', 'LoyaltyPoints', "Conversion"]
hd = df.copy()

plt.figure(figsize=(8,6))
sns.heatmap(hd[historical_conv_features].corr(numeric_only=True),annot=True,cmap='Blues')
plt.title('Correlations of Historical Data and Conversion',fontsize=15)
plt.show()
```

Correlations of Historical Data and Conversion



Insights

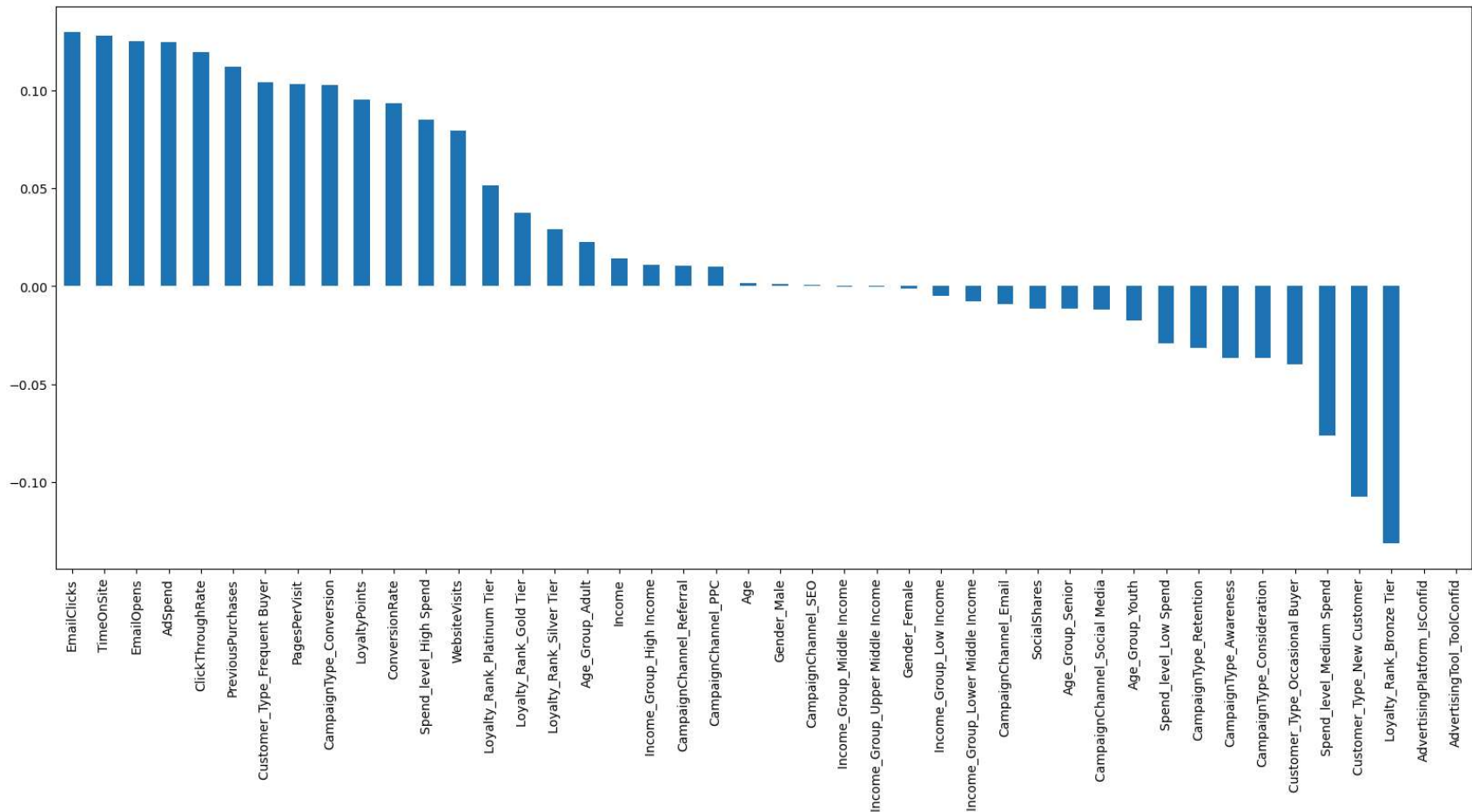
- **Positive Correlation with Previous Purchases:** PreviousPurchases shows a positive correlation with Conversion (approximately 0.112). This indicates that customers who have made more previous purchases are more likely to convert, suggesting customer loyalty or a proven inclination to buy.
- **Positive Correlation with Loyalty Points:** LoyaltyPoints also exhibits a positive correlation with Conversion (approximately 0.108). This implies that customers with higher loyalty points, which usually accrue from past engagement and purchases, are more prone to converting.

- **Reinforcement of Customer Value:** Both historical metrics highlight that established customer relationships, as evidenced by past purchases and accumulated loyalty, are significant indicators of future conversion likelihood. Businesses should consider targeting these segments with relevant campaigns.

5.5 Correlation with Every Features

```
In [34]: # Get Correaltion of conversion with other variables
df1 = df_dummies.corr()['Conversion'].sort_values(ascending=False)
plt.figure(figsize=(20,8))
df1.drop('Conversion').plot(kind= 'bar')
```

Out[34]: <Axes: >



Insights:

Most Positively Correlated Features with Conversion:

- **High Engagement Metrics:** `TimeOnSite` (0.130), `EmailClicks` (0.130), and `EmailOpens` (0.125) show the strongest positive correlations. This strongly suggests that higher customer engagement with the website and email campaigns directly leads to a higher likelihood of conversion.
- **Ad Performance & Prior Behavior:** `AdSpend` (0.125), `ClickThroughRate` (0.120), `PreviousPurchases` (0.112), and `LoyaltyPoints` (0.108) also exhibit significant positive correlations. This indicates that effective ad campaigns, prior customer buying behavior, and accumulated loyalty are strong indicators of conversion.
- **Specific Campaign & Customer Types:** `Customer_Type_Frequent Buyer` (0.104), `CampaignType_Conversion` (0.103), and `PagesPerVisit` (0.103) also have positive correlations, highlighting the effectiveness of conversion-focused campaigns and the value of frequent buyers.

Least Correlated (or Negatively Correlated) Features with Conversion:

- **Demographic Features:** Most demographic features (`Age` , `Gender` , `Income` , and their respective groups) show very weak correlations (typically below 0.06), indicating that these factors alone are not strong direct predictors of conversion.
- **Advertising Platform/Tool:** `AdvertisingPlatform_IsConfid` and `AdvertisingTool_ToolConfid` have correlations close to zero, suggesting they do not significantly influence conversion, likely due to them being confidential and having only one unique value (i.e., they don't vary across customers).
- **Campaign Type Indicators for Non-Conversion:** `CampaignType_Awareness` , `CampaignType_Consideration` , and `CampaignType_Retention` show negative correlations, meaning that these campaign types are less directly associated with immediate conversion compared to 'Conversion' type campaigns. Similarly, `Customer_Type_New Customer` and `Customer_Type_Occasional Buyer` have negative correlations, reinforcing that established customers are more likely to convert.

Overall Takeaway: Conversion is primarily driven by active customer engagement (time on site, email interactions), effective ad performance, and historical purchasing behavior. Demographic factors have a much weaker linear relationship with conversion.

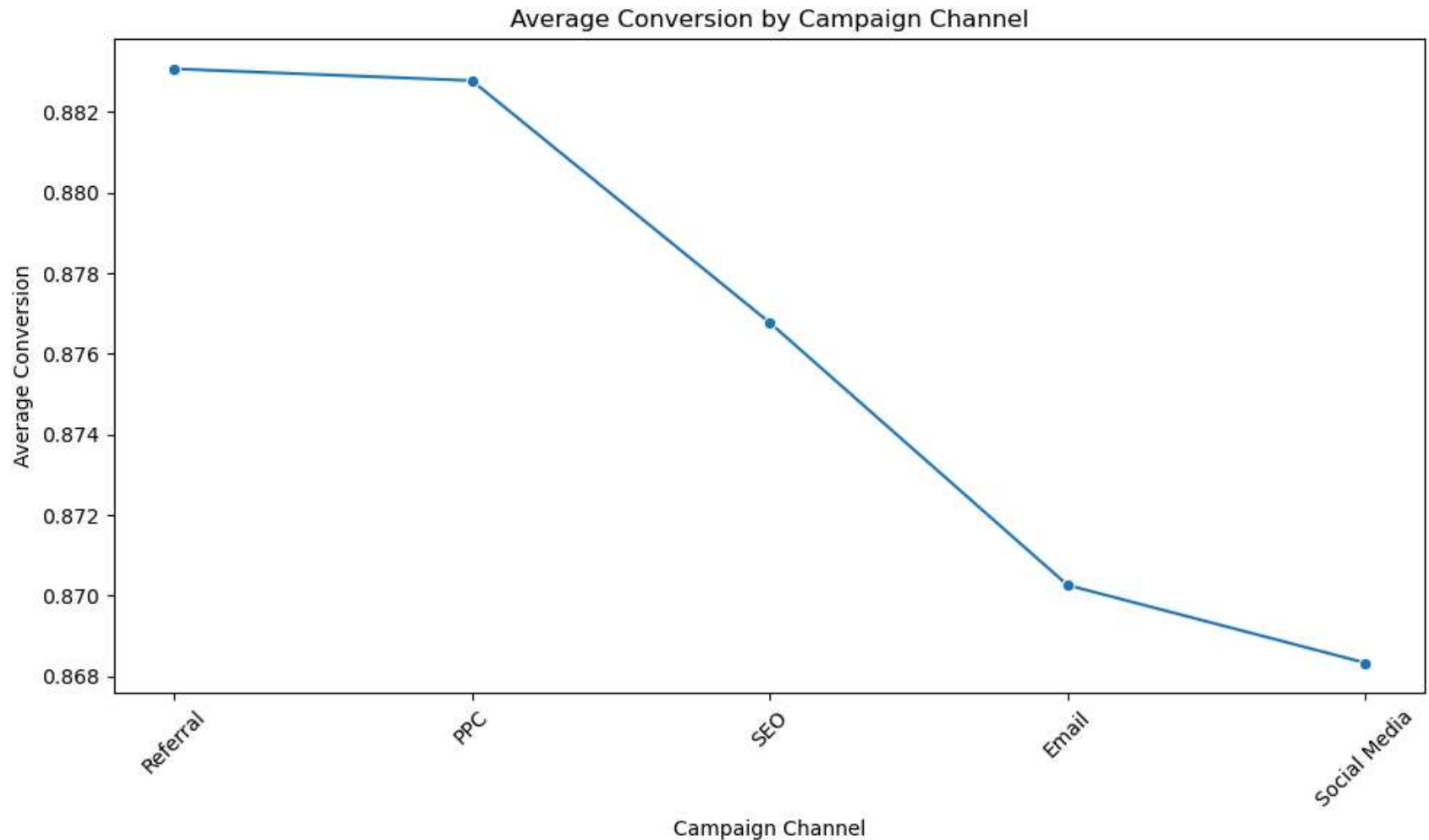
5.6 Question And Answers

Q1: How does Conversion rate vary by CampaignChannel?

Reason: Identifies high-performing channels and informs feature importance and targeting strategies for the model.

```
In [35]: # Calculate the average conversion rate for each Campaign Channel
conversion_rate_by_channel = df.groupby('CampaignChannel')['Conversion'].mean().sort_values(ascending=False)
```

```
# Create a Line plot
plt.figure(figsize=(10, 6))
sns.lineplot(x=conversion_rate_by_channel.index, y=conversion_rate_by_channel.values, marker='o')
plt.title('Average Conversion by Campaign Channel')
plt.xlabel('Campaign Channel')
plt.ylabel('Average Conversion')
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```



- Referral and PPC channels demonstrate the highest average conversion rates, suggesting they are the most effective in driving conversions.
- SEO conversion rates are slightly lower than Referral and PPC but still perform well.

- Email and Social Media campaigns show the lowest average conversion rates among the channels. However, it's important to note that these rates are **still** relatively high overall, indicating that while they might not be the top performers, they **still** contribute significantly to conversions.

Q2: What is the relationship between AdSpend and Conversion, and does it differ by CampaignType?

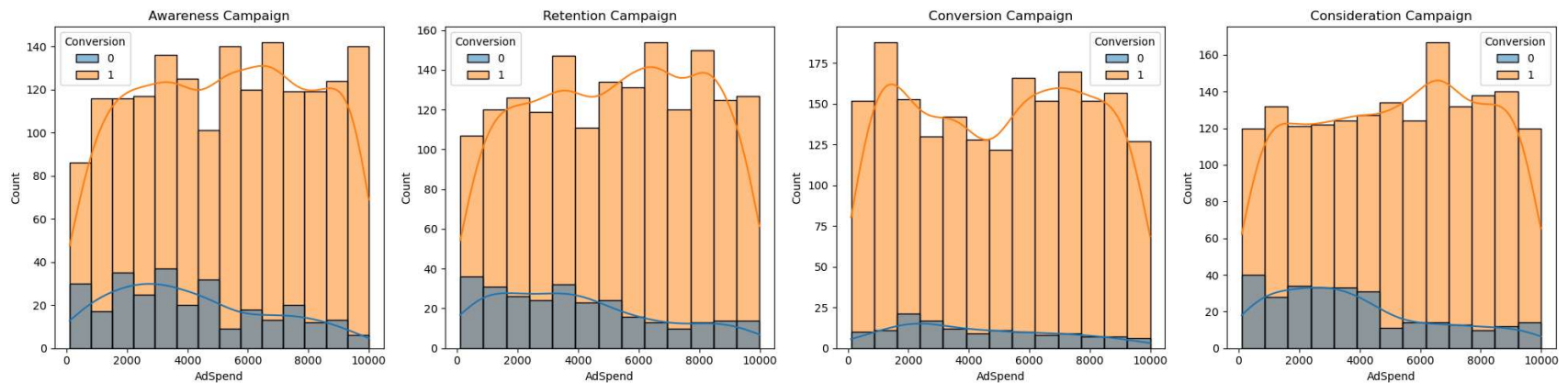
Reason: Helps detect diminishing returns, optimal spend ranges, and interaction effects between spend and campaign objective.

```
In [36]: # Create subplots for each campaign type
campaign_types = df['CampaignType'].unique()
n_campaigns = len(campaign_types)

fig, axes = plt.subplots(1, n_campaigns, figsize=(5*n_campaigns, 5))

for i, campaign in enumerate(campaign_types):
    sns.histplot(data=df[df.CampaignType == campaign], x='AdSpend', kde=True, hue='Conversion', ax=axes[i])
    axes[i].set_title(f'{campaign} Campaign')
    axes[i].set_xlabel('AdSpend')
    axes[i].set_ylabel('Count')

plt.tight_layout()
plt.show()
```



- **Awareness and Consideration Campaigns:** Conversions are distributed across various AdSpend levels, with some effectiveness noted even at lower AdSpend. This suggests that these campaign types may not rely solely on high AdSpend for their objectives.

- **Retention Campaigns:** Similar to the above, conversions are spread across different AdSpend values, indicating that consistent engagement, rather than just high spending, contributes to customer loyalty and repeat purchases.
- **Conversion Campaigns:** In these campaigns, AdSpend appears to have a more concentrated impact. While conversions occur across all AdSpend levels, there might be optimal AdSpend ranges that more effectively drive conversions.
- **General Observation:** Conversions happen at various AdSpend levels across all campaign types. This implies that AdSpend is one factor among many that influence conversions, and its impact is intertwined with other campaign elements.

Q3: Which engagement metrics are most predictive of Conversion?

Reason: Prioritizes behavioral features and guides which signals to engineer (e.g., thresholds, interactions).

```
In [40]: # Calculate the number of rows and columns for subplots
excluded_cols = ['CustomerID', 'Age', 'Gender', 'Income', 'AdSpend', 'CampaignType', 'CampaignChannel', 'ConversionRate',
numerical_cols = [col for col in numeric_features if col not in excluded_cols]

n_features = len(numerical_cols)
n_cols = 3 # You can adjust the number of columns as needed
n_rows = (n_features + n_cols - 1) // n_cols # Calculate the number of rows

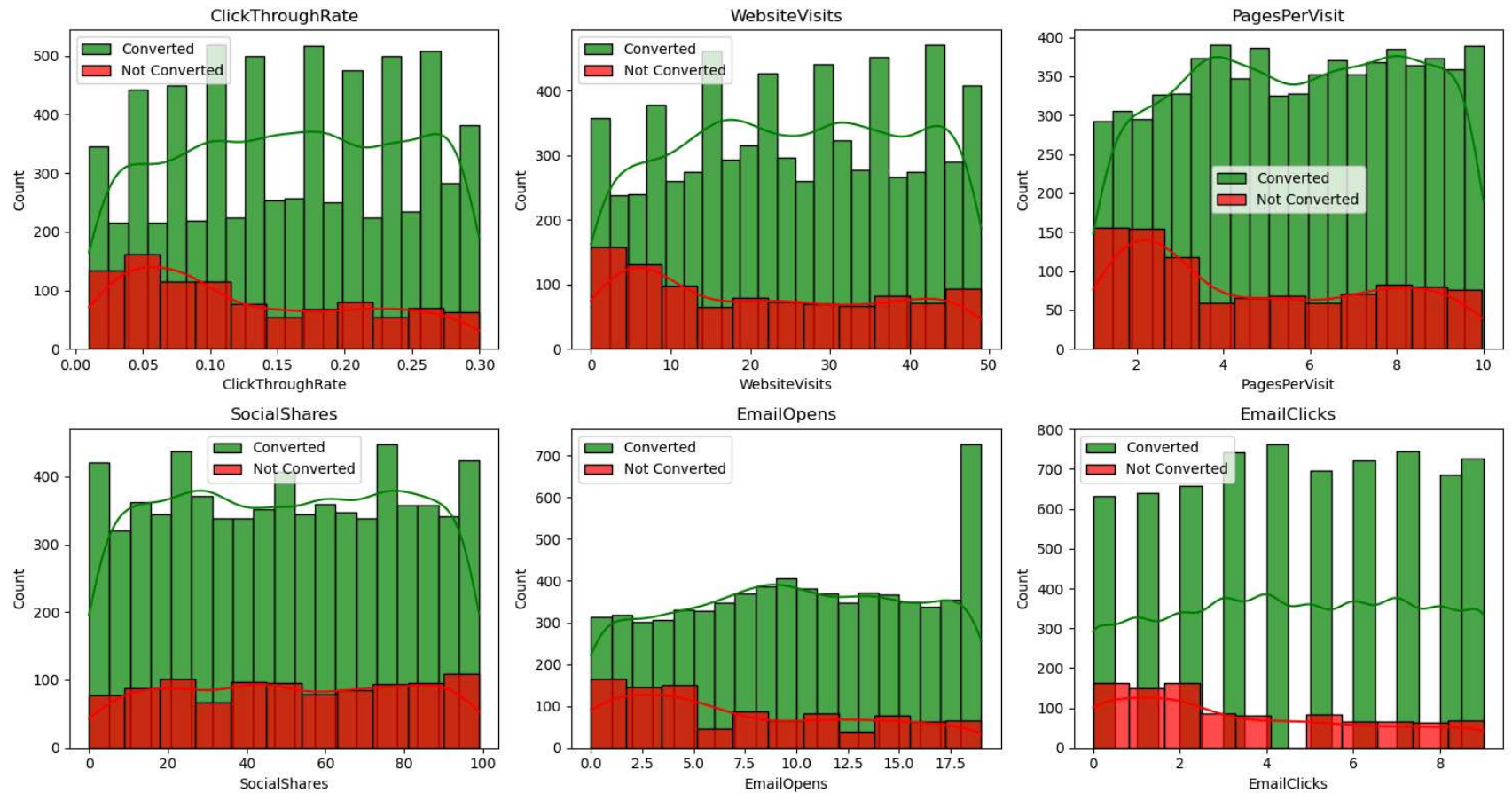
# Create figure and subplots
fig, axes = plt.subplots(n_rows, n_cols, figsize=(n_cols * 5, n_rows * 4), squeeze=False)

# Flatten axes array for easier iteration
axes = axes.flatten()

# Plot histograms for each numerical feature, separated by conversion status
for i, col in enumerate(numerical_cols):
    sns.histplot(data=df[df["Conversion"] == 1], x=col, color="green", label="Converted", alpha=0.7, ax=axes[i], kde=True)
    sns.histplot(data=df[df["Conversion"] == 0], x=col, color="red", label="Not Converted", alpha=0.7, ax=axes[i], kde=True)
    axes[i].set_title(col)
    axes[i].set_xlabel(col)
    axes[i].set_ylabel("Count")
    axes[i].legend()

# Hide any unused subplots
for j in range(i + 1, len(axes)):
    axes[j].set_visible(False)

plt.tight_layout()
plt.show()
```



- **ClickThroughRate:** The distributions show that converted customers tend to have higher click-through rates, indicating that a more appealing initial interaction is often a precursor to conversion.
- **WebsiteVisits:** Converted customers generally exhibit a higher frequency of website visits, suggesting that repeated exposure and interaction with the site contribute significantly to the conversion process.
- **PagesPerVisit:** Customers who convert tend to browse more pages per visit, implying a deeper level of interest and exploration of the offerings.
- **TimeOnSite:** Similarly, converted customers spend more time on the website, which is a strong indicator of engagement and consideration.
- **SocialShares:** While both groups engage in social sharing, the plots suggest a slight lean towards more shares among converted customers, potentially reflecting satisfaction or advocacy.

- **EmailOpens and EmailClicks:** There's a clear distinction here: converted customers show significantly higher numbers of both email opens and clicks. This strongly emphasizes the critical role of email campaign engagement in driving conversions.

Key Insights on Customer Conversion Factors

Based on the comprehensive analysis of the digital marketing campaign dataset, the following key factors significantly influence customer conversion:

1. Engagement is Paramount

- **TimeOnSite, EmailClicks, and EmailOpens** show the strongest positive correlations with conversion. Customers who spend more time on the website and actively engage with email campaigns (opening and clicking links) are highly likely to convert.
- **WebsiteVisits, PagesPerVisit, and SocialShares** also have positive correlations, indicating that frequent visits, deeper exploration of website content, and social advocacy contribute to conversion.

2. Historical Behavior is a Strong Predictor

- **PreviousPurchases** and **LoyaltyPoints** are positively correlated with conversion. Customers with a history of purchases and accumulated loyalty points demonstrate a higher propensity to convert, suggesting the importance of retaining existing customers.

3. Campaign Effectiveness Varies by Channel and Type

- **Referral and PPC channels** exhibit the highest average conversion rates, making them highly effective for driving conversions.
- **"Conversion" type campaigns** are, unsurprisingly, the most effective in generating conversions, followed by Awareness and Consideration campaigns which also contribute significantly.
- **AdSpend** shows a positive correlation with conversion, indicating that strategic investment in campaigns yields results, though its impact can vary by campaign type.

4. Demographics Have Limited Direct Predictive Power (Linearly)

- Individual demographic attributes like **Age, Gender, and Income** (and their respective groups) show very weak linear correlations with conversion. While these factors might influence conversion through more complex interactions, their direct impact as standalone predictors is minimal.
- The higher absolute number of conversions in certain demographic groups (e.g., 'Adult' age group, 'Female' gender, 'Upper Middle Income') is largely due to their larger representation in the dataset rather than a significantly higher conversion rate.

within those groups.

5. Data Completeness and Consistency

- The dataset is robust with **no missing values or duplicate records**, which strengthens the reliability of the insights derived.
- Initial data type corrections for `ClickThroughRate` and `ConversionRate` ensured accurate numerical analysis.

Actionable Recommendations to Improve Conversion Rates

Based on these insights, here are actionable recommendations to optimize digital marketing efforts and boost conversion rates:

1. Prioritize and Enhance Engagement Strategies

- **Optimize Website Experience:** Focus on improving website navigation, content relevance, and call-to-action placement to encourage longer `TimeOnSite` and more `PagesPerVisit`.
- **Strengthen Email Marketing:** Develop highly engaging email campaigns with compelling content and clear CTAs to maximize `EmailOpens` and `EmailClicks`. Implement A/B testing on subject lines, content, and send times.
- **Encourage Social Sharing:** Integrate social sharing options more prominently on high-value content or product pages to leverage word-of-mouth marketing and increase `SocialShares`.

2. Leverage Historical Customer Data for Targeted Marketing

- **Implement Loyalty Programs:** Further invest in and promote loyalty programs to reward `PreviousPurchases` and accumulated `LoyaltyPoints`. Target these loyal customers with exclusive offers and personalized communications.
- **Develop Retention Campaigns:** Create specific campaigns aimed at customers with multiple past purchases to encourage repeat business, as they show a high likelihood of converting again.

3. Optimize Campaign Channel and Type Allocation

- **Allocate More Budget to High-Performing Channels:** Increase investment and focus on **Referral and PPC channels**, given their demonstrated higher conversion rates. Continuously monitor performance to identify the most cost-effective strategies within these channels.
- **Strategically Design Campaign Types:** While "Conversion" campaigns are crucial for immediate results, ensure a balanced approach with "Awareness" and "Consideration" campaigns to nurture the pipeline. Tailor ad spend and content for each campaign type's objective.
- **Analyze AdSpend Efficiency:** Conduct deeper analysis into the optimal `AdSpend` ranges for different `CampaignType`s to maximize ROAS (Return on Advertising Spend).

4. Refine Customer Segmentation Beyond Basic Demographics

- **Focus on Behavioral Segmentation:** Shift primary targeting efforts from broad demographic categories to behavioral segments based on engagement metrics (e.g., high `TimeOnSite` , high `EmailClicks`), purchase history, and loyalty status. This will allow for more precise and effective audience targeting.
- **Utilize Demographic Data for Context, Not Primary Targeting:** While demographic data might not be strong direct predictors, use them to provide context for behavioral segments (e.g., tailoring messaging style for different age groups within a high-engagement segment).

By implementing these recommendations, businesses can strategically enhance their digital marketing campaigns, leading to improved customer engagement and higher conversion rates.

We have completed our EDA section of this Project now let's go towards the Machine learning Model Training & Building. To Check out the Notebook file for the Machine learning phase click / copy & paste this link :

https://github.com/Rishabh90501/DMC_Conversion/blob/b0a5c79f3ad74e1d3b6191bf3d812f8528e36cf3/notebook/DMC_Machine_Learning_Model_Training_Building.ipynb

